

100ASK_IMX6ULL

ARM Cortex-A7

Embedded Development Platform

Peripheral module Manual

Rev. 1.0

2020/03/25



0755-86200561



support@100ask.net



Shenzhen, Guangdong, China



Shenzhen 100ask Technology Co.

■ 注意事项与售后维修

1. 注意事项

- 使用产品之前，请仔细阅读本手册，并妥善保管，以备将来参考；
- 请注意和遵循标注在产品上的所有警示和指引信息；
- 请使用配套电源适配器，以保证电压、电流的稳定；
- 请在凉爽、干燥、清洁的地方使用本产品；
- 请勿在冷热交替环境中使用本产品，避免结露损坏元器件；
- 请勿在湿气过重、温度过高或过低环境中使用本产品，使用时注意产品的通风；
- 请勿将任何液体泼溅在本产品上，禁止使用有机溶剂或腐蚀性液体清洗本产品；
- 请勿在多尘、脏乱的环境中使用本产品，如果长期不使用，请包装好本产品；
- 请勿在振动过大的环境中使用，任何跌落、敲打或剧烈晃动都可能损坏线路及元器件；
- 请勿在通电情况下，插拔核心板及外围模块(特别是串口模块)；
- 请勿自行维修、拆解本产品，如产品出现故障应及时联系本公司进行维修；
- 请勿自行修改或使用未经授权的配件，由此造成的损坏将不予保修；

2. 售后维修

1) 保修期限

- 底板、核心板：三个月（非人为损坏）
- 显示屏：七天（非人为损坏）

2) 保修说明

- 7天内：产品（底板、核心板、屏幕）非人为损坏，本公司免费更换/维修，并承担来回运费；
- 7天至3个月内：底板、核心板非人为损坏，本公司免费维修，并承担来回运费（屏幕不提供维修）；
- 3个月至1年：底板、核心板非人为损坏或人为轻微损坏，只收更换元器件费用，免费维修，买家承担来回运费；
- 起始时间以快递签收日为准；

3) 联系方式

官方网站：www.100ask.net

淘宝网站：100ask.taobao.com

地 址：广东省深圳市龙岗区布吉南湾街道平吉大道建昇大厦 B1505

联系人：售后维修部

电 话：0755-86200561

邮 编：518114

邮寄须知：保修期限内，寄回本产品请预先垫付邮费，公司不接收任何到付快递。

技术支持与开发定制

1. 技术支持范围

- 1) 本公司提供的各类开发软件的安装，入门使用，环境搭建；
- 2) 本公司提供的所有裸机代码的烧写验证；
- 3) 本公司发布的操作系统的编译、烧写；
- 4) 本公司发布产品的工控板、模块的硬件原理；
- 5) 本公司发布的各种外设模块驱动及源码；
- 6) 本公司发布的配套手册在使用过程中遇到的问题；
- 7) 本公司产品的故障诊断及售后维修服务；

2. 技术讨论范围

由于嵌入式系统知识范围广泛，涉猎种类繁多，我们无法保证对各种问题都能一一解答，以下内容无法供技术支持，只能提供建议。

- 1) 本公司发布的教程之外的知识；
- 2) 非本公司发布的 U-Boot、Linux 内核的编译和移植；
- 3) 非本公司发布的工控板的各类驱动支持；
- 4) 非本公司发布的外设模块的硬件原理和驱动设计；

3. 技术支持方式

- 1) 官方论坛发帖提问(推荐): bbs.100ask.net
- 2) 官方淘宝通过阿里旺旺咨询: 100ask.taobao.com
- 3) QQ 群咨询 (QQ 群号咨询淘宝客服, 需提供淘宝购买订单号验证加入);
- 4) 技术支持邮箱: support@100ask.net
- 5) 联系电话: 0755-86200561

4. 技术支持时间

星期一到星期五;上午 9:00—12:00;下午 14:00—17:30;

公司按照国家法定节假日安排休息,在此期间无法提供技术支持,请将问题发送至技术支持邮箱或在论坛对应板块发帖,我们将在工作日尽快给您回复。

5. 投诉和建议

如果您对我们有不满意或者建议,可发送邮件到 support@100ask.net 进行反馈,也可拨打 0755-86200561 取得联系,我们将不断改进。

6. 定制开发服务

本公司提供嵌入式操作系统底层驱动、硬件板卡的有偿定制开发服务,以缩短您的产品开发周期。请将需求发送邮件到 support@100ask.net。

资料获取与后续更新

1. 资料的获取

1) 百度网盘下载

百度网盘里面有本产品的所有配套资料，包括原理图、发布的 U-Boot、内核镜像和源码、所需的开发软件、工具等等。

进入 www.100ask.net，在导航栏选择“资料下载”，点击“到百度网盘下载”，跳转到百度网盘后，找到对应的文件夹即可。

2) 视频配套教程

后续会为该工控板录制一套裸机、Linux 驱动、应用的配套付费教学视频，有需要的客户可以进入官方淘宝 100ask.taobao.com 选购。

3) 维基百科教程

维基百科里面会有视频配套的笔记，进入 wiki.100ask.net，选择对应的板块查看。

2. 后续更新

后续文档、视频等资料的更新，为了确保您的资料是最新状态，请密切关注我们的动态，我们将会通过微信公众号和 QQ 群公告推送，购买了本产品的客户请添加 QQ 群（QQ 群号咨询淘宝客服，需提供淘宝购买订单号验证加入）或关注微信公众号。



版权声明

百问科技©2020

深圳百问网科技有限公司版权所有，并保留对本手册及声明的一切权力。

未得到本公司的书面许可，任何单位和个人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途

更新记录

类别	100ASK_IMX6ULL 系列文档
文档名	100ASK_IMX6ULL Peripheral module Manual
当前版本	1.0
适用型号	100ASK_IMX6ULL
编辑	百问科技文档编辑团队
审核	韦东山

修改日志		
版本	修改时间	更改说明
1.0	2020.03.25	初始版本，黄成

目录

注意事项与售后维修.....	I
技术支持与开发定制.....	II
资料获取与后续更新.....	III
版权声明.....	IV
更新记录.....	V
目录.....	1
前言.....	2
第一章 百问网模块介绍.....	3
1.1 模块的分类.....	3
1.2 通用模块.....	4
1.3 扩展板介绍.....	6
1.4 直接测试.....	7
1.5 源码编译测试.....	9
第二章 模块测试.....	12
2.1 LED 灯.....	12
2.2 Keyboard 按键.....	13
2.3 IrDA 红外遥控.....	14
2.4 DHT11 温湿度传感器.....	16
2.5 DS18B20 温度传感器.....	17
2.6 SR501 人体红外感应.....	18
2.7 SR04 超声波测距.....	20
2.8 Motor 步进电机.....	21
2.9 OLED 显示.....	23
2.10 DAC 数模转换.....	25
2.11 ADXL345 三轴加速度计.....	27
2.12 EEPROM 存储模块.....	28
2.13 GPS 模块.....	29
2.14 ADC 模数转换.....	31
2.15 OLED 硬件监控.....	34
附录一 底板扩展接口和扩展板原理图.....	37
附录二 模块原理图.....	38
附录三 按键映射表.....	42

前言

本手册主要目的是介绍 100ASK_IMX6ULL 支持的模块，以及这些模块的简单功能测试，帮助用户快速验证收到的模块功能是否正常。

模块的测试用例是直接移植的《100ASK_IMX6ULL 开发板 Arduino 使用手册》里的 Arduino 应用程序库，以接口分类，介绍了 LED 灯、按键、红外遥控、温湿度传感器、人体红外检测、超声波测距、步进电机控制、OLED 显示、三轴加速度计、数模/模数转换、GPS 模块等的基本使用，最后再示例了一个实战项目供读者参考。如果读者对这些模块的底层实现感兴趣，可以通过前面的资料获取的方式，免费/收费获取裸机、Linux 驱动的底层实现教学视频、配套文档。

目前本手册涉及的模块都是常见的 GPIO、SPI、I2C、UART 接口等，后续还会推出 RS485、CAN、USB 摄像头等其它模块。

第一章 百问网模块介绍

1.1 模块的分类

以 100ASK_IMX6ULL 为例，开发板由核心板和底板组成。

核心板是一个最小系统，包含 CPU、RAM、Flash 和电源芯片等必须的芯片，以及引出没有使用的引脚。

底板是将核心板的引脚转换大家熟悉的接口，比如网口、USB 口、LCD 接口、TTL 电平的串口等。此外还包含少量的模块，比如 WIFI/Bluetooth 等。

用户想要什么扩展功能，就需要在对应的接口接上对应的模块。比如需要显示功能，就需要接上 LCD 模块；需要 4G 网络，就需要接上 4G 模块；需要检测环境温度，就需要接上温度传感器模块。每种模块都有一个接口，LCD 是 RGB，4G 模块是 PCIE，温度传感器是 GPIO。

百问网把一些使用常见、简单的协议接口（GPIO、I2C、SPI、UART）的模块做成了通用模块，配合扩展板，使模块可以在百问网多个系列开发板上可以直接使用。

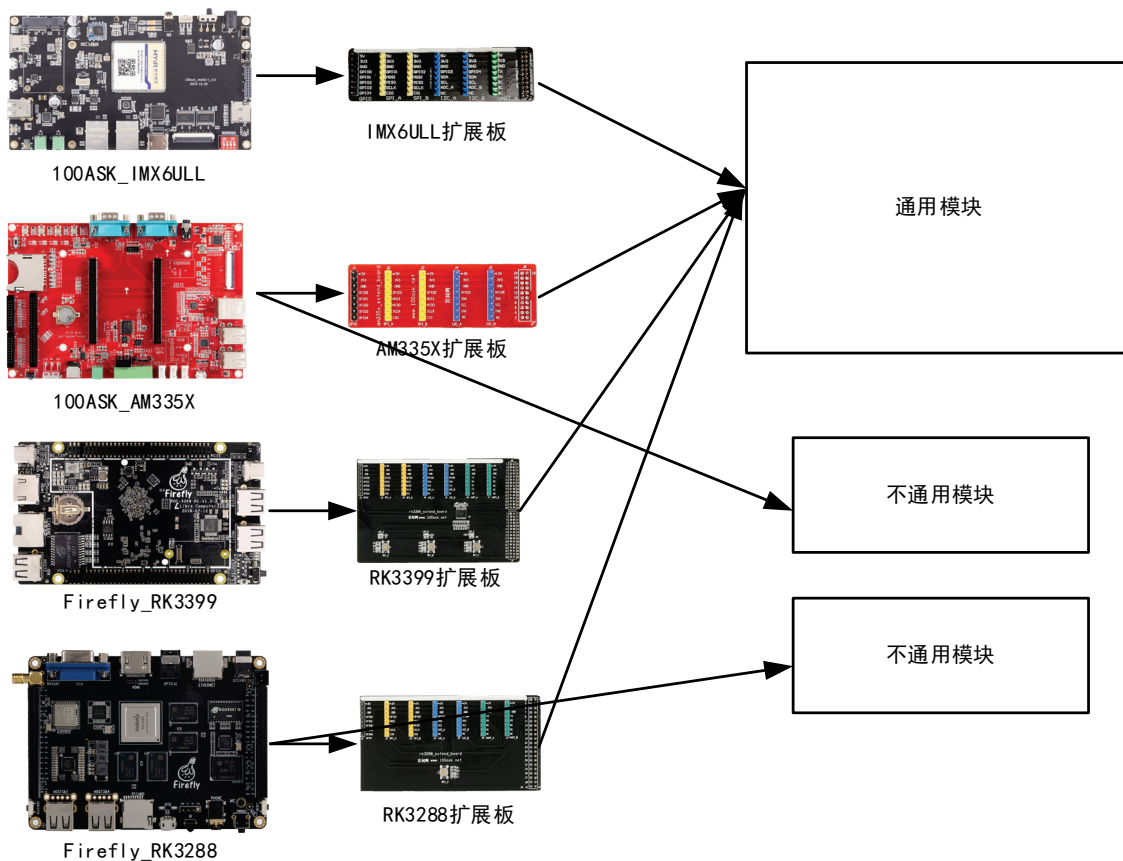
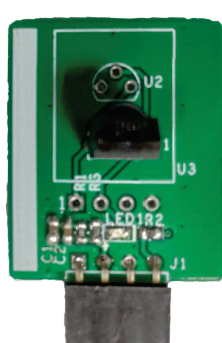


图 1.1.1 开发板、扩展板、模块之间的关系

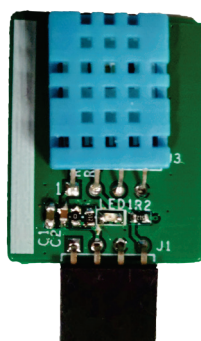
本手册目前主要介绍通用模块，后续一些不通用的模块，比如 RS485 转 CAN 模块，等设计完后再更新文档介绍。

1.2 通用模块

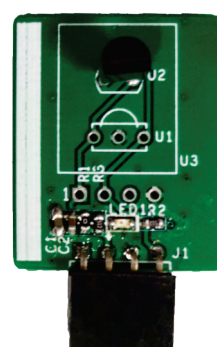
目前设计好的通用模块有十余个，后续可能会继续生产更多模块，现有模块如下图所示。



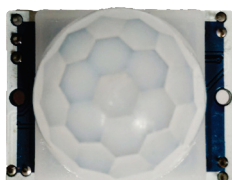
IrDA红外遥控
接收模块



DHT11温湿度
传感器



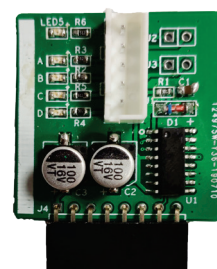
DS18B20温度
传感器



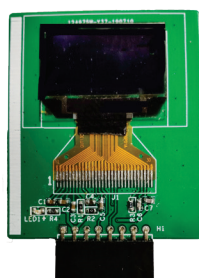
SR501人体红外
感应模块



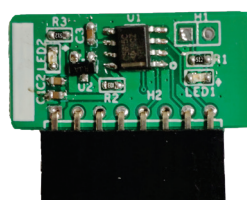
SR04超声波测距
模块



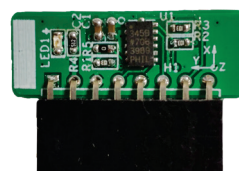
步进电机驱动
模块



OLED显示
模块



DAC模数转换
模块



ADXL345三轴
加速计模块



EEPROM/光敏电阻
模块



GPS定位
模块

图 1.2.1 通用模块实物图

测试程序支持的模块清单列表：

GPIO接口			
编号	模块名	接口	备注
1	LED模块	底板板载	LED2 USER
2	Keyboard按键模块	底板板载	USER1、USER2
3	IrDA红外遥控	接扩展板	一个GPIO、配遥控器
4	DHT11温湿度模块	接扩展板	一个GPIO
5	DS18B20温度模块	接扩展板	一个GPIO
6	SR501人体红外模块	接扩展板	一个GPIO
7	SR04超声波测距模块	接扩展板	两个GPIO
8	Motor步进电机驱动板	接扩展板	四个GPIO、配步进电机
SPI接口			
编号	模块名	接口	备注
1	OLED显示屏	接扩展板	复用GPIO
2	DAC数模转换	接扩展板	——
3	ADXL345三轴传感器	接扩展板	复用GPIO
I2C接口			
编号	模块名	接口	备注
1	EEPROM存储模块	接扩展板	与光敏电阻在一个模块
其它接口			
编号	模块名	接口	备注
1	GPS定位模块	接扩展板	——
2	ADC模数转换	底板板载	通过光敏电阻展示效果

1.3 扩展板介绍

每个开发板的扩展接口引脚顺序各有不同，因此还需要扩展板转接成统一的接口。100ASK_IMX6ULL 扩展板示意图如图 1.3.1 所示。

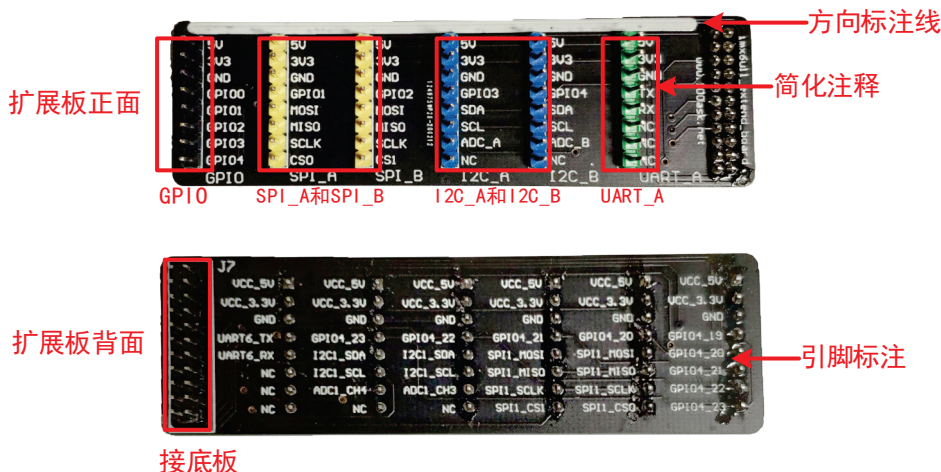


图 1.3.1 100ASK_IMX6ULL 扩展板实物图

扩展板正面从左到右依次是一列 GPIO 接口、两列 SPI 接口 (SPI_A 和 SPI_B)、两列 I2C 接口 (I2C_A 和 I2C_B)，一列 UART 接口 (UART_A)，其中 I2C 接口所在列还包含 ADC 接口 (ADC_A 和 ADC_B)。每列接口都提供 5V、3V3 电源，以便支持工作电压不同的模块。

扩展板正面文字注释为**简化注释**，后面文档、代码如果没有特殊说明，都是使用该简化注释。上方有根白色的方向标注线用于方向指引，配合模块上的白色标注线，两根在同边即可。

扩展板背面文字注释为 100ASK_IMX6ULL 原理图上的**引脚标注**，参考附录一，供想深入学习的用户参考。

使用扩展板的 GPIO 时要注意：

- ① 对于只需要一个 GPIO 的模块，可以修改相应代码后，插在五个排针座中的任意一个的上半部分，如图 1.3.2 所示的 GPIO0、GPIO1、GPIO2、GPIO3、GPIO4；
- ② 对于步进电机驱动板这种需要两个及以上 GPIO 的模块，只能插在最左边的黑色排针 GPIO0 上；
- ③ 同一个 GPIO，不能同时用于多个模块。比如不能同时使用电机驱动板和 OLED 模块。因为电机驱动板占用了 GPIO0~3；而 OLED 模块本来可以接在 SPI_A 或 SPI_B，但所需的 GPIO1~2 被电机驱动板占用了。

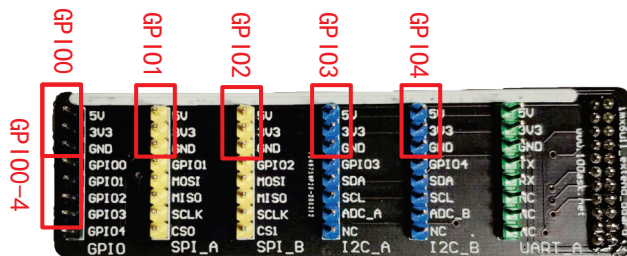


图 1.3.2 100ASK_IMX6ULL 扩展板 GPIO 介绍

扩展板插在底板的“J5 Camera & Extend”上(在 TF 卡槽附近)，方向如图 1.3.3 所示。

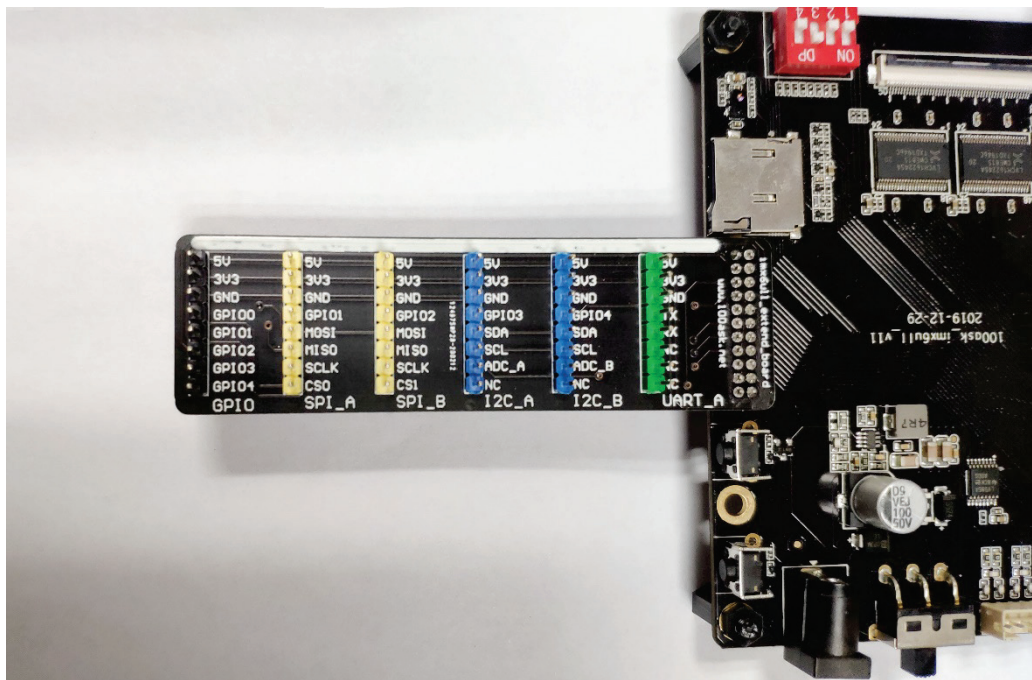


图 1.3.3 扩展板和底板连接实物图

1.4 直接测试

测试程序提供**二进制文件**和**源码文件**。

如果用户只想测试模块功能，可以直接下载二进制文件，上传到开发板，接好模块，运行即可看到效果。

如果用户想修改测试程序，或者想研究下测试程序的实现，可以下载源码，在 Ubuntu 上交叉编译后上传运行。

本小节介绍一些基本配置和如何下载二进制文件。

1.4.1 一些基本配置

在运行测试程序前，需要修改设备树和驱动开机自动加载脚本。

修改设备树是因为开发板只有一个系统状态灯，该灯出厂时在设备树里定义为心跳灯，供系统使用。但测试用例中，不少会用到该 LED 来展示结果，因此需要修改设备树。

开发板上电，连接网线，确保能连接互联网的情况下，依次执行以下命令：

```
[root@imx6ull:~]# cp /boot/100ask_imx6ull-14x14.dtb /boot/100ask_imx6ull-14x14.dtb_bak
[root@imx6ull:~]# wget --no-check-certificate
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/
imx6ullModule/dts/100ask_imx6ull-14x14.dtb
[root@imx6ull:~]# mv 100ask_imx6ull-14x14.dtb /boot/100ask_imx6ull-14x14.dtb
```

该命令首先备份/boot 目录下的原设备树文件，然后下载新设备树文件，最后将下载新设备树文件移动到/boot 目录下完成替换。

如果用户开发板不能连接互联网，可以复制前面命令里的链接(https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/imx6ullModule/dts/100ask_imx6ull-14x14.dtb)，粘贴到浏览器下载，再通过 FTP/SFTP 传到开发板的/boot 目录下。

如果用户使用 buildroot，更新到了最新的状态，并重新编译烧写了整个系统，那么可能已经添加了驱动开机自动加载脚本，可执行以下命令测试是否添加了驱动开机自动加载：

```
[root@imx6ull:~]# grep "modprobe" /etc/init.d/S09modload
```

如果返回结果为如下结果，表示已经添加：

```
modprobe 8723bu
modprobe 100ask_ds18b20
modprobe 100ask_dht11
modprobe 100ask_rc-nec
modprobe 100ask_irda
modprobe 100ask_spidev
modprobe 100ask_adxl345-spi
```

如果返回结果为如下结果，表示没有添加：

```
modprobe 8723bu.ko
```

此时需要用户执行：

```
[root@imx6ull:~]# wget --no-check-certificate
https://weidongshan.coding.net/p/buildroot_2019.02/d/buildroot_2019.02/git/raw/master/board/100ask/nxp-imx6ull/rootfs-overlay/etc/init.d/S09modload
[root@imx6ull:~]# chmod u+x S09modload && mv S09modload /etc/init.d/S09modload
```

同样的，用户的开发板如果无法连接互联网，依旧可以复制前面的链接到浏览器下载，并移动到相应的目录里替换。后面类似的下载，不再赘述。

最后，如果用户使用的开发板**不带 LCD 显示屏**，还需要执行以下命令替换触摸屏定义脚本：

```
[root@imx6ull:~]# cp /etc/init.d/S07hmi /etc/init.d/S07hmi_bak
[root@imx6ull:~]# wget --no-check-certificate
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/imx6ullModule/init.d/S07hmi
[root@imx6ull:~]# chmod u+x S07hmi && mv S07hmi /etc/init.d/S07hmi
```

替换设备树和相关脚本后，执行以下命令重启开发板：

```
[root@imx6ull:~]# reboot
```

1.4.2 下载二进制文件

以闪烁 LED 为例，启动开发板，连接互联网后，执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/imx6ullModule/app_bin/LED/Blink
[root@imx6ull:~]# chmod u+x Blink
```

再执行：

```
[root@imx6ull:~]# ./Blink
```

即可看到开发板 LED 灯闪烁。

如果需要下载其它测试文件，需要对应修改链接结尾的模块名称和测试程序名字，比如下载按键测试文件，需要把链接末尾的“/LED/Blink”改为“Keyboard/readKeyboard”，下载命令对应为：

```
[root@imx6ull:~]# wget --no-check-certificate
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/
imx6ullModule/app_bin/Keyboard/readKeyboard
[root@imx6ull:~]# chmod u+x readKeyboard
```

再执行：

```
[root@imx6ull:~]# ./readKeyboard
```

此时按下开发板的 USER_KEY1 或 USER_KEY2，即可在串口上看到打印结果。

所有的测试文件如下表所示，供用户参考：

GPIO接口			
编号	模块名	测试程序名	备注
1	LED	Blink	LED2 USER交替闪烁
2	Keyboard	readKeyboard	读取板载按键值
3	Keyboard	turnOffLED	按键控制LED关闭
4	IrDA	readIRDA	读取红外遥控器按键
5	IrDA	turnOnLED	红外遥控器控制LED打开
6	DHT11	readDHT11	读取温湿度传感器模块值
7	DS18B20	readDS18B20	读取温度传感器模块值
8	SR501	getSignal	获取人体感应信号
9	SR04	getDistance	获取距离传感器距离
10	Motor	stepperMotor	步进电机测试
SPI接口			
编号	模块名	测试程序名	备注
1	OLED	oledDisplay	OLED显示模块测试
2	DAC	breathingLight	DAC控制LED亮度
3	ADXL345	detectionTouch	检测触碰示例
4	ADXL345	readXYZData	读取XYZ轴数据
I2C接口			
编号	模块名	测试程序名	备注
1	EEPROM	readWriteArray	EEPROM读写数组
2	EEPROM	readWriteString	EEPROM读写字符串
其它接口			
编号	模块名	测试程序名	备注
1	GPS	printfRawData	打印GPS模块原始数据
2	GPS	printfAnalyzeData	打印GPS分析处理后的数据
3	ADC	readBrightness	读取光照亮度
模块组合			
1	Project	OLEDMonitor	OLED显示系统信息和温度

1.5 源码编译测试

1.5.1 安装Ubuntu虚拟机

用户需要先安装 Ubuntu 虚拟机，该部分参考《100ask_imx6ull 开发板用户手册-入门篇.pdf》。该文档在下载方法如下。

打开 <http://wiki.100ask.net/Download>，拉到网页下面，找到“100ask_imx6ull 开发板 BSP 包”，点开对应的百度链接。

打开“00_UserManual”目录，即可看到《100ask_imx6ull 开发板用户手册-入门篇.pdf》。下载该文档，只看“第2章 搭建开发环境”。如果使用我们配置好环境的 Ubuntu 镜像，看“2.1 安装并允许 Ubuntu 系统”即可。如果用户想自行搭建 Ubuntu 并设置环境，看“2.2 自行配置并安装 Ubuntu 虚拟机系统”。

另外，“2.4 安装、使用 MobaXterm”、“2.5 使用 FileZilla 新建 SFTP”、“2.6 开发板串口驱动安装”，也需要用户根据需求参考。

1.5.2 下载、编译测试源码

假设用户以及完成以上内容，进入 Ubuntu 终端，执行以下命令。

```
[root@imx6ull:~]# cd ~ && mkdir imx6ull_app && cd imx6ull_app
[root@imx6ull:~]# git clone https://github.com/100askTeam/arduino-imx6ull.git
[root@imx6ull:~]# cd arduino-imx6ull
[root@imx6ull:~]# wget --no-check-certificate
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/
imx6ullModule/imx6ull_complete.sh
[root@imx6ull:~]# chmod u+x imx6ull_complete.sh
```

此时，源码文件在“libraries/”目录里，“imx6ull_complete.sh”为编译脚本，执行如下命令可以查看支持编译的模块和测试程序名字：

```
book@100ask:~/imx6ull_app/arduino-imx6ull$ ./imx6ull_complete.sh list
ADC/readBrightness
ADXL345/detectionTouch
ADXL345/readXYZData
DAC/breathingLight
DHT11/readDHT11
DS18B20/readDS18B20
EEPROM/readWriteArray
EEPROM/readWriteString
GPS/printfAnalyzeData
GPS/printfRawData
IrDA/readIRDA
IrDA/turnOnLED
Keyboard/readKeyboard
Keyboard/turnOffLED
LED/Blink
Motor/stepperMotor
OLED/oledDisplay
Project/OLEDMonitor
SR04/getDistance
SR501/getSignal
```

左边是模块名称，右边是测试程序名字，如果要编译某个程序，执行脚本加上测试名字即可，比如编译 LED 的测试程序：

```
book@100ask:~/imx6ull_app/arduino-imx6ull$ ./imx6ull_complete.sh readBrightness
-----Compile successfully!-----
The file information:
  text    data    bss    dec    hex filename
  4060    396    152    4608    1200 imx6ull_app
Copy imx6ull_app to 100ASK_IMX6ULL board, eg:
scp imx6ull_app root@192.168.1.35:/root/
-----
```

编译成功后，会生成“imx6ull_app”文件，如果开发板与 Ubuntu 在同一个局域网，执行：

```
book@100ask:~/imx6ull_app/arduino-imx6ull$ scp imx6ull_app root@192.168.1.35:/root/
```

即可把“imx6ull_app”复制到开发板的“/root”目录，这里“192.168.1.35”是开发板的 IP，需要修改为用户自己开发板的 IP。

确保开发板按前面“1.4.1 一些基本配置”完成配置，在开发板“/root”目录下执行以下命令即可看到效果：

```
[root@imx6ull:~]# ./imx6ull_app
```


第二章 模块测试

目前为 Arduino 提供了 10 余个模块的示例代码。按接口类型，分成了四类：

- ① GPIO 接口
- ② SPI 接口
- ③ I2C 接口
- ④ 其它接口（UART 接口、ADC 等）

每类接口里面有几个模块。比如 GPIO 接口下，有 LED 灯、Keyboard 按键、IrDA 红外、DHT11 温湿度传感器等，其中 LED 和 Keyboard 是底板自带的，其它需要用户外接。

每个模块里面，至少有一个示例程序。比如 LED 灯模块下有一个 Bink 示例程序实现 LED 灯交替闪烁；Keyboard 按键模块下有两个示例程序：readKeyboard 程序实现读取按键，turnOffLED 程序实使用按键关闭 LED 灯。

后续会持续更新代码修复漏洞或增添模块，用户按照前面“1.4 或 1.5 小节”下载最新的即可。

2.1 LED 灯

2.1.1 硬件连接

LED 灯是开发板底板板载的，无需硬件连接。

LED 灯在开发板左上脚，分别是“LED1 PWR”、“LED2 USR”、“LED3 4G”，其中只有 LED2 用户才能控制。

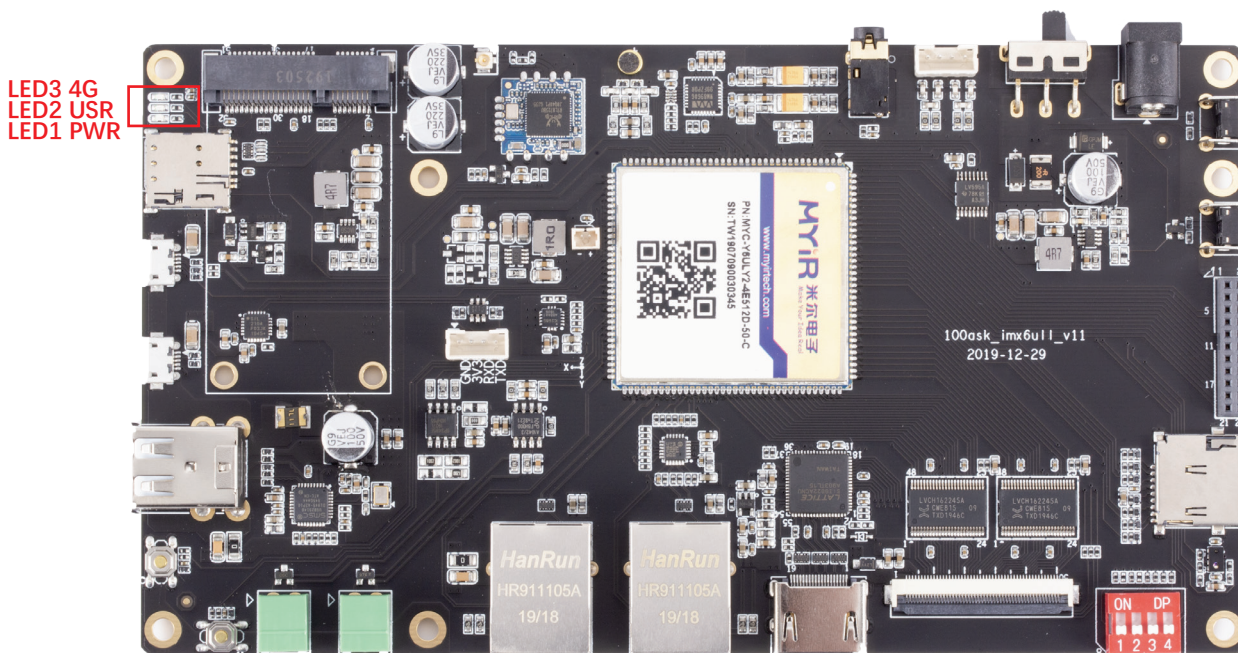


图 2.1.1 100ASK_IMX6ULL 的 LED 位置

2.1.2 下载、运行、效果

启动开发板，连接互联网后，执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/
```

```
imx6ullModule/app_bin/LED/Blink  
[root@imx6ull:~]# chmod u+x Blink
```

执行以下命令运行程序：

```
[root@imx6ull:~]# ./Blink
```

即可看到开发板 LED2 间隔一秒交替闪烁。

注意：在终端里按下组合键“Ctrl”+“C”，可终止当前正在运行的程序。

2.2 Keyboard 按键

2.2.1 硬件连接

Keyboard 也是开发板底板板载的，无需硬件连接。

按键在开发板左下脚，分别是“KEY1 USR1”、“KEY2 USR2”，两个按键都可由用户使用。

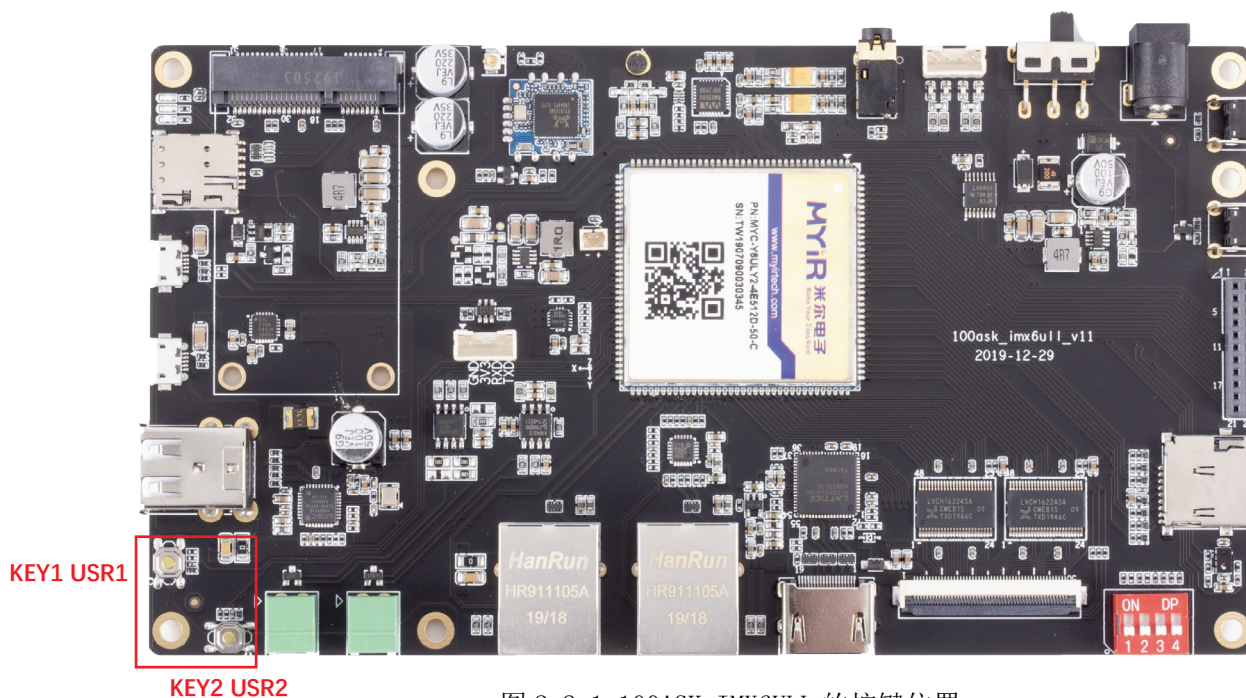


图 2.2.1 100ASK_IMX6ULL 的按键位置

2.2.2 下载、运行、效果

启动开发板，连接互联网后，执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/Keyboard/readKeyboard  
[root@imx6ull:~]# chmod u+x readKeyboard
```

执行以下命令运行程序，分别按下“KEY1 USR1”、“KEY2 USR2”，打印如下：

```
[root@imx6ull:~]# ./readKeyboard  
Code:2 Value:1  
Code:2 Value:1
```



```
Code:2 Value:0
Code:2 Value:0

Code:3 Value:1
Code:3 Value:1
Code:3 Value:0
Code:3 Value:0
```

可以看到每次按键，分别打印了两组数据，每组打印了两个值：“code”和“value”。

“code”用来表示哪一个按键被按下，“value”表示当前按键的状态(1表示按下，0表示松开，2表示长按)。

KEY1在Linux输入子系统对应的code值为2，KEY1对应的code值为3。按键对应的code值参考附录三按键映射表，当有按键发生，就可以根据code值来判断是哪一个被按下。

除“readKeyboard”示例外，还提供了一个“turnOffLED”的示例，它实现通过KEY1关闭LED2的功能，用户可以自行上传程序体验。

2.3 IrDA 红外遥控

红外遥控被广泛应用于家用电器、工业控制和智能仪器系统中，像我们熟知的有电视机盒子遥控器、空调遥控器。红外遥控器系统分为发送端和接收端，如图2.3.1所示。

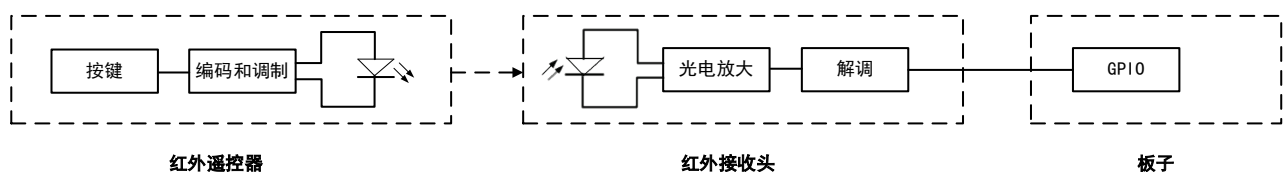


图 2.3.1 红外遥控系统原理示意图

发送端就是红外遥控器，上面有许多按键，当我们按下遥控器按键时，遥控器内部电路会进行编码和调制，再通过红外发射头，将信号以肉眼不可见的红外线发射出去。红外线虽然肉眼不可见，但可以通过手机摄像头看到，常用该方法检查遥控器是否正常工作。

接收端是一个红外接收头，收到红外信号后，内部电路会进行信号放大和解调，再将数据传给板子上的GPIO，板子收到数据后再解码才能确定是哪个按键被按下。



图 2.3.2 红外遥控器和红外接收头实物图

2.3.1 硬件连接

首先将 IrDA 红外接收模块接到扩展板上，为了防止用户接错方向，模块和扩展板都有一条长白线，连接时需要将这段对齐。将红外接收模块接在扩展板的 GPIO 接口 0，如图 2.3.3 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

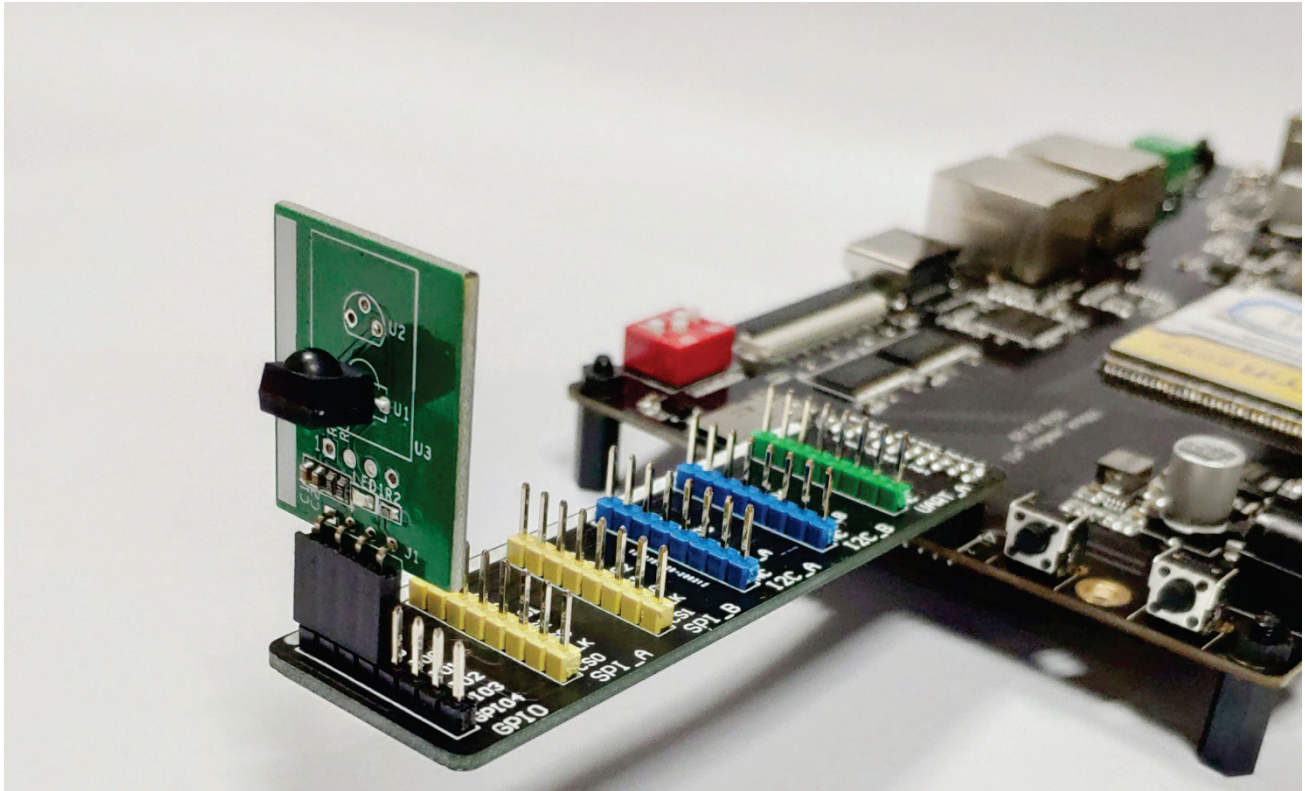


图 2.3.3 红外接收模块、扩展板实物连接图

正常连接好后，红外接收模块上的电源指示灯(红色)会点亮。然后准备好配套的红外遥控器，如果是第一次使用红外遥控器，要先取出电池上的隔离薄膜。

2.3.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/IrDA/readIRDA  
[root@imx6ull:~]# chmod u+x readIRDA
```

执行以下命令运行程序，按下红外遥控器上的按键 1，再松开，可以看到串口打印如下：

```
[root@imx6ull:~]# ./readIRDA  
Code:2 Value:1  
Code:2 Value:0
```

可以看到打印的“code”和“value”。“code”用来表示哪一个按键被按下，“value”表示当前按键的状态，和前面的 Keyboard 含义一样。红外遥控器所有按键对应的 code 值参考附录三按键映射表。

除了“readIRDA”示例，还提供了一个“turnOnLED”示例，实现按下 KEY_1 或者长按 KEY_1 点亮 LED_A。

2.4 DHT11 温湿度传感器

DHT11 温湿度传感器是一个与温度、湿度相关的传感器。它除了能测量温度以外，还能测量湿度，比如市面上一些空气加湿器，会测量空气中湿度，再根据测量结果决定是否继续加湿。

DHT11 测量温度的精度为 $\pm 2^{\circ}\text{C}$ ，检测范围为 -20°C ~ 60°C 。湿度的精度为 $\pm 5\%\text{RH}$ ，检测范围为 $5\%\text{RH}$ ~ $95\%\text{RH}$ 。

2.4.1 硬件连接

将 DHT11 接到 GPIO 接口 0，如图 2.4.1 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

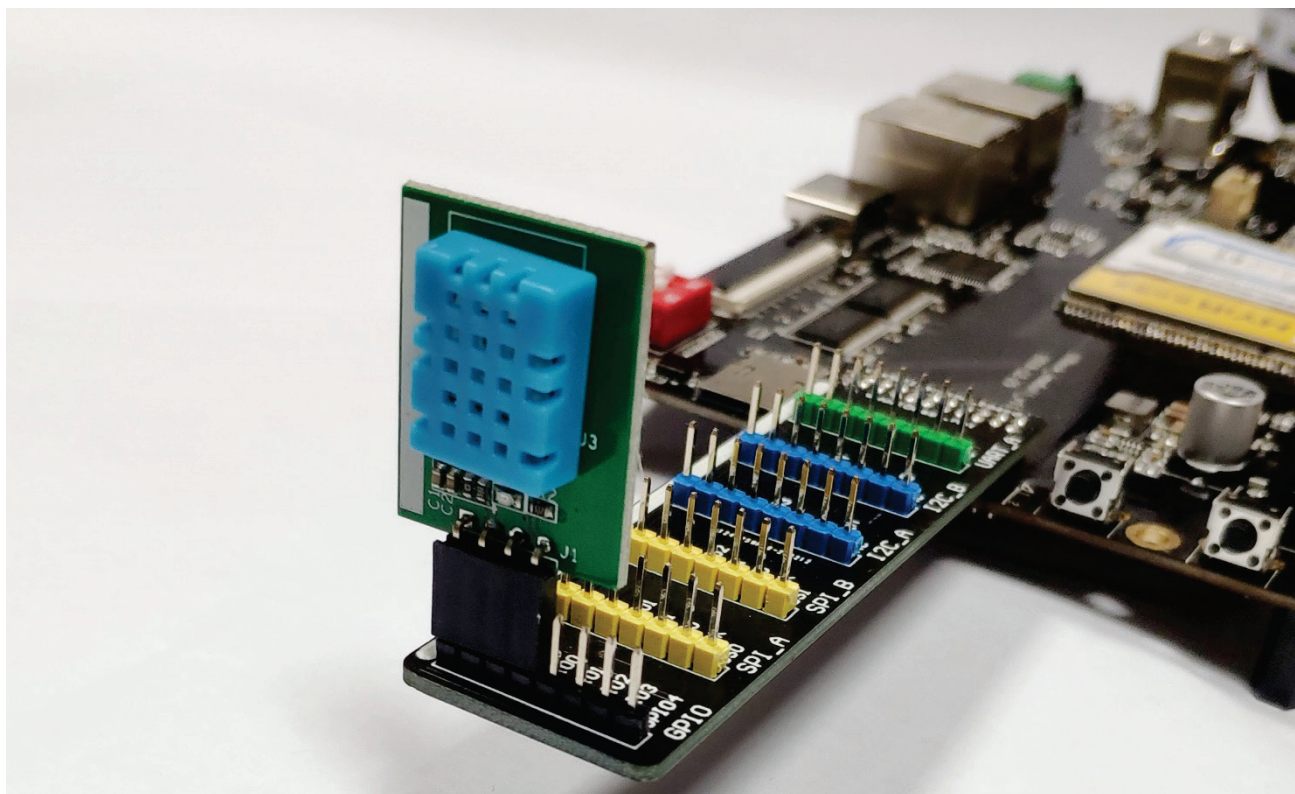


图 2.4.1 DHT11 温湿度传感器、扩展板实物连接图

2.4.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/DHT11/readDHT11  
[root@imx6ull:~]# chmod u+x readDHT11
```

执行以下命令运行程序，即可看到当前温度、湿度打印信息如下：

```
[root@imx6ull:~]# ./readDHT11  
Temperature: 23'C  
Humdity: 76%
```


2.5 DS18B20 温度传感器

DS18B20 一款用途更广泛的温度传感器，默认精度是 0.0625℃，测量范围为-55℃~ 125℃。

2.5.1 硬件连接

将 DS18B20 接到 GPIO 接口 0，如图 2.5.1 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

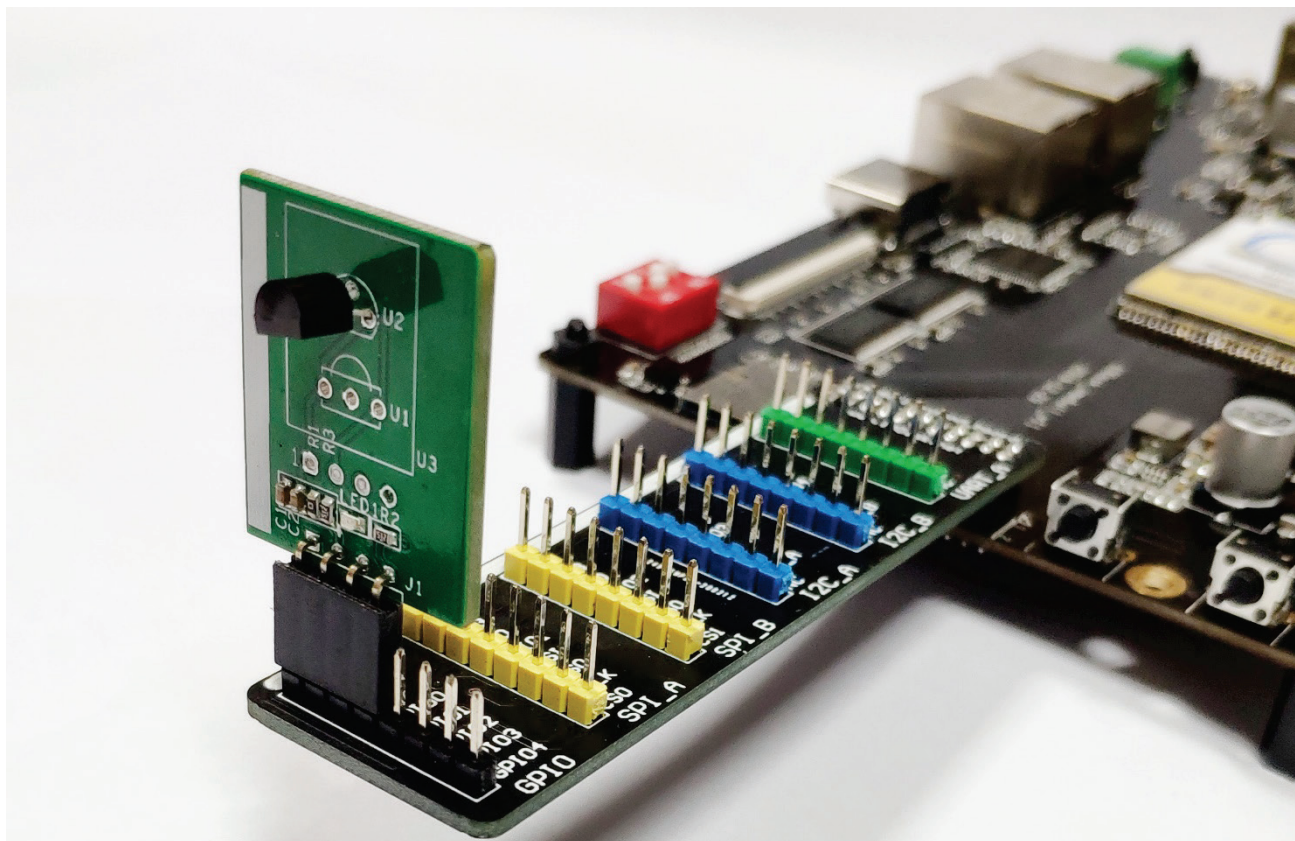


图 2.5.1 DS18B20 温度传感器、扩展板实物连接图

2.5.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/DS18B20/readDS18B20  
[root@imx6ull:~]# chmod u+x readDS18B20
```

执行以下命令运行程序，即可看到当前温度如下：

```
[root@imx6ull:~]# ./readDS18B20  
SerialNum: 2171720318250  
Temperature: 21.5625  
FamilyCode: 40
```

CRC: 3

除了温度以外，还显示了 DS18B20 唯一序列号“SerialNum”、产品系列号“FamilyCode”，以及用于 CRC 校验的“CRC”，只需要关心温度值“Temperature”即可。

2.6 SR501 人体红外感应

热释红外传感器是一种能够检测人或动物发射的红外线而输出电信号的传感器。广泛应用于各种自动化控制装置中。比如常见的楼道自动开关、防盗报警等。

人体红外传感器如下图所示，上面有两个可调电阻，一个用于控制感应距离，一个用于控制延时时间。感应距离好理解，即感应人体的距离，能够最大达到 7 米，延时时间是指感应到红外信号后，输出高电平的时间，可调为 0.3 秒至 10 分钟。**建议将延时时间旋钮逆时针旋转调至最低，以减少前后两次检测的反应时长。**



图 2.6.1 人体红外感应模块左、右侧面图

2.6.1 硬件连接

随模块一起的还有一根定制杜邦线，这根杜邦线一端为 3Pin，一端为 4Pin。3Pin 端接模块，4Pin 端接扩展板。注意红色线统一表示接电源，黑色线统一表示接地。定制杜邦线与模块连接图如下。

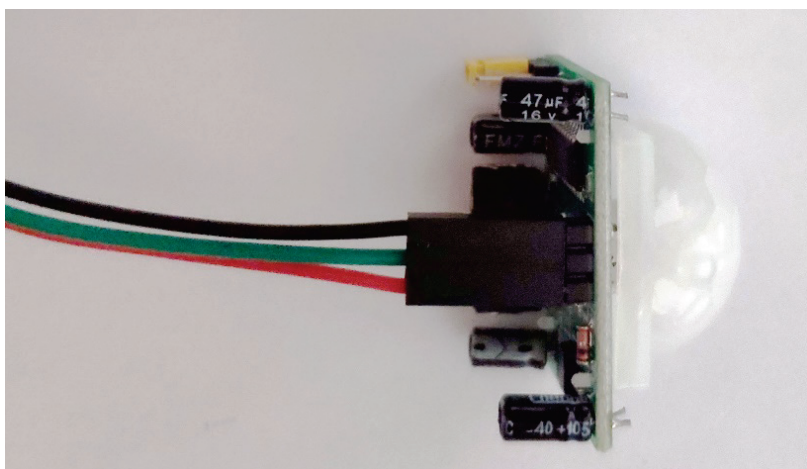


图 2.6.2 人体红外感应模块与定制杜邦线连接实物图

接着将杜邦线另一端接到扩展板的 GPIO 接口 0，注意电源(红色线)的方向，如图 2.6.3 所示。

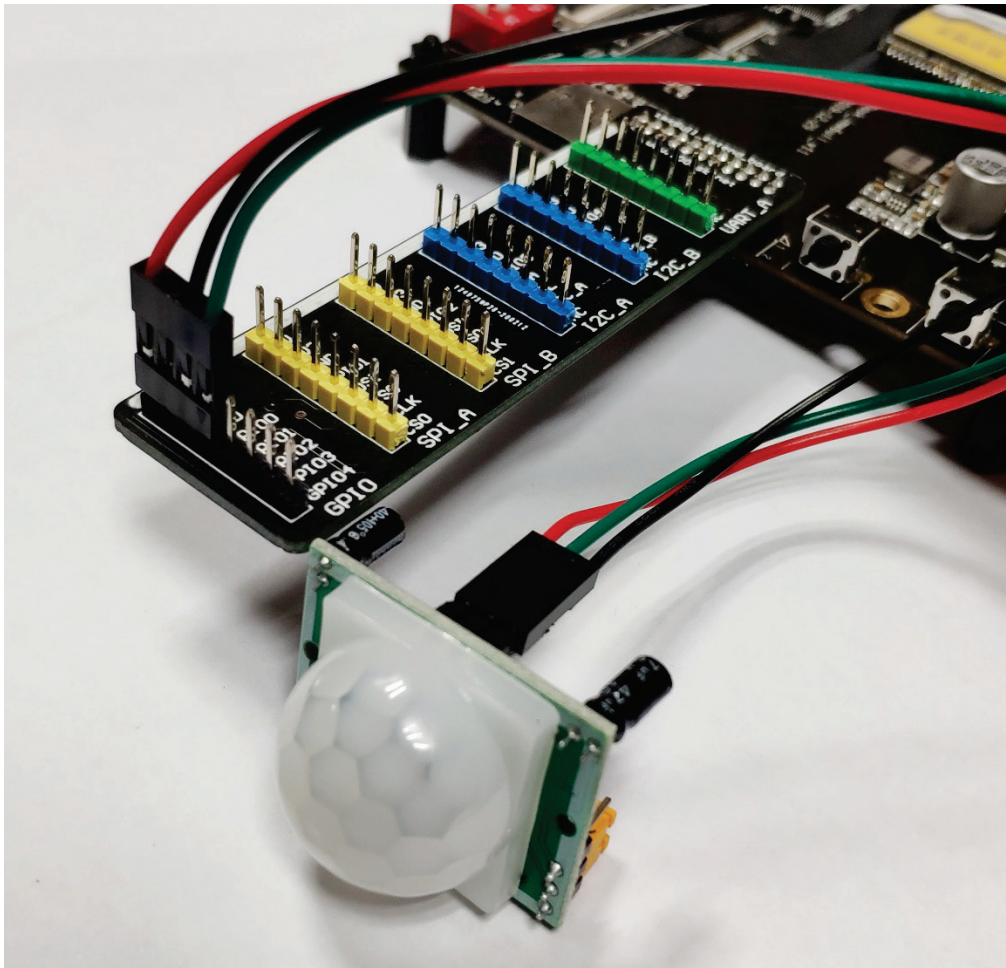


图 2.6.2 SR501 人体红外模块、扩展板实物连接图

2.6.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/SR501/getSignal  
[root@imx6ull:~]# chmod u+x getSignal
```

执行以下命令运行程序，再将手靠近 SR501 模块，可以看到类似如下打印：

```
[root@imx6ull:~]# ./getSignal  
No one detected!  
No one detected!  
No one detected!  
No one detected!  
No one detected!  
No one detected!  
Detected someone!  
Detected someone!
```


Detected someone!

注意：模块在每次检测后，会持续该状态一段时间，因此每次用手靠近触发后，再等一阵子才会执行“getSignal()”函数进行下一次判断。

2.7 SR04 超声波测距

超声波测距模块是利用超声波来测距。模块先发送超声波，然后接收反射回来的超声波，由反射经历的时间和声音的传播速度 340m/s，计算得出距离。

2.7.1 硬件连接

由于超声波模块需要外接两个 GPIO 引脚，因此只能接在 GPIO0 所在接口上，使用配套杜邦线可直接相连，注意电源(红线)方向，实物连接如下图 2.7.1 所示。

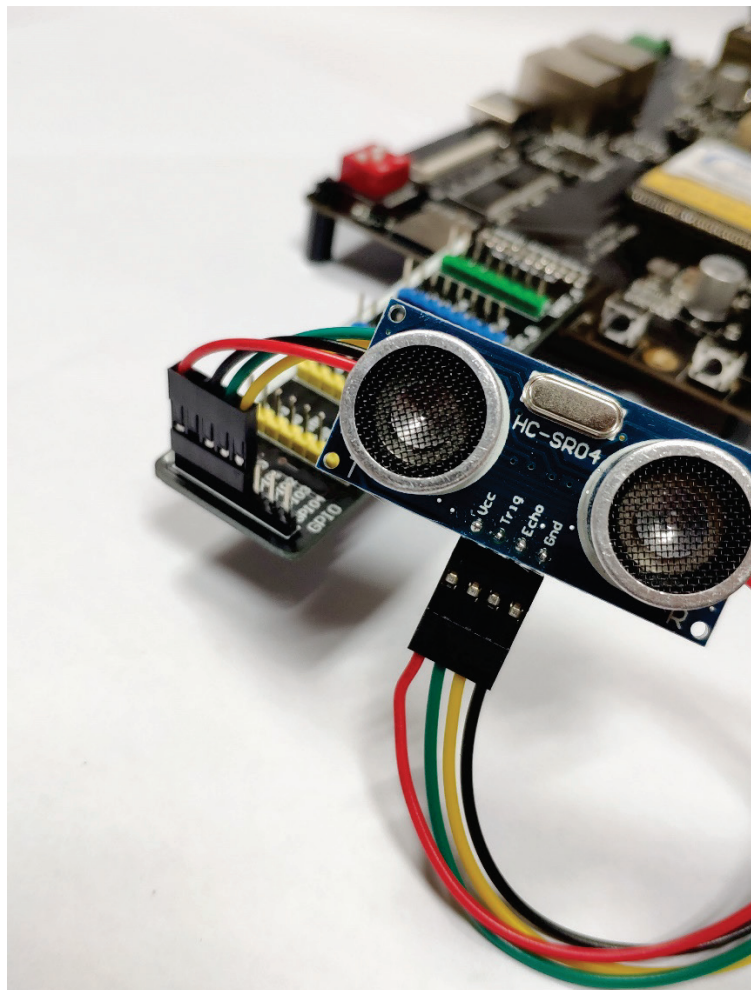


图 2.7.1 SR04 超声波测距模块、扩展板实物连接图

2.7.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/SR04/getDistance  
[root@imx6ull:~]# chmod u+x getDistance
```

执行以下命令运行程序，再将手由远到近靠近模块，可以看到如下类似打印：

```
[root@imx6ull:~]# ./getDistance  
distance: 28cm  
distance: 24cm  
distance: 21cm  
distance: 21cm  
distance: 17cm  
distance: 14cm  
distance: 10cm  
distance: 9cm  
distance: 10cm  
distance: 7cm  
distance: 7cm  
distance: 3cm  
distance: 3cm
```

注意：模块的盲区距离为 2cm，太靠近模块是测量不出来的。

2.8 Motor 步进电机

步进电机是一种将电脉冲转化为角位移的执行机构。通俗的讲，就是当步进驱动器接收到一个脉冲信号，它就驱动步进电机按设定的方向转动一个固定的角度。因此可以通过控制脉冲个数来控制角位移量，从而达到准确定位的目的，同时可以通过控制脉冲频率来控制电机转动的速度，从而达到调速的目的。

2.8.1 硬件连接

将步进电机与步进电机驱动模块相连，控制步进电机需要四个 GPIO，因此只能直接 GPIO0 上，连接实物图如 2.8.1 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

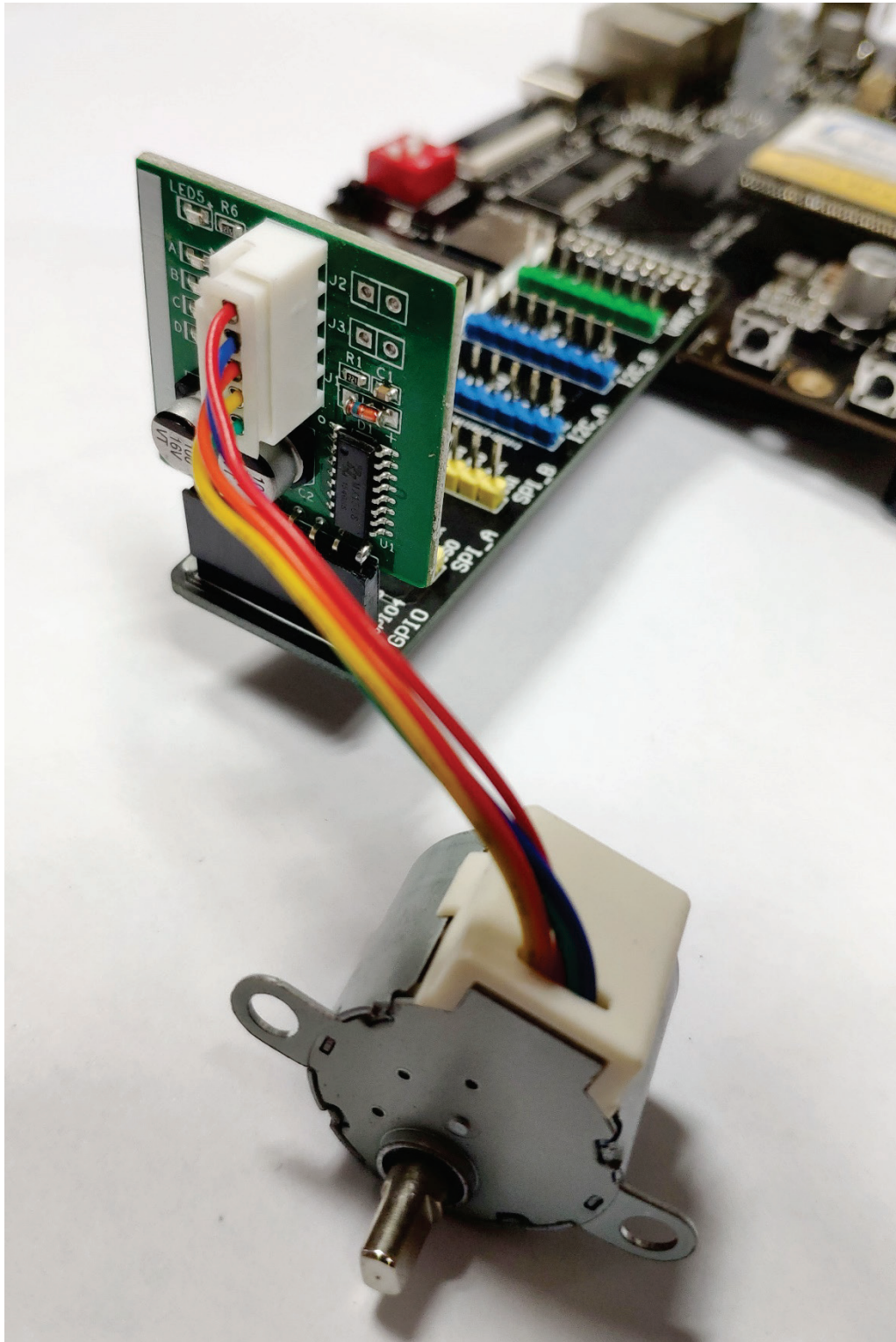


图 2.8.1 步进电机、步进电机驱动板、扩展板实物连接图

2.8.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/
```



```
imx6ullModule/app_bin/Motor/stepperMotor  
[root@imx6ull:~]# chmod u+x stepperMotor
```

执行以下命令运行程序，可以看到步进电机缓慢的正转一圈，再反转一圈，最后停止：

```
[root@imx6ull:~]# ./stepperMotor
```

注意：程序发生意外退出时，可能 GPIO 最后仍有输出，会导致电机慢慢发热烧毁。因此在不使用电机时，应该取下步进电机或取下步进电机驱动板。

2.9 OLED 显示

板子适配一块 7 寸的 LCD，它的接口、操作都比较复杂。这里介绍 SPI 接口的 OLDE，它的显示空间虽然比较小，但成本低、功耗低，可以用来显示一些重要信息，或者作为副屏显示。

2.9.1 硬件连接

OLED 使用 SPI 接口，将其插在 SPI_A 上，如图 2.9.1 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

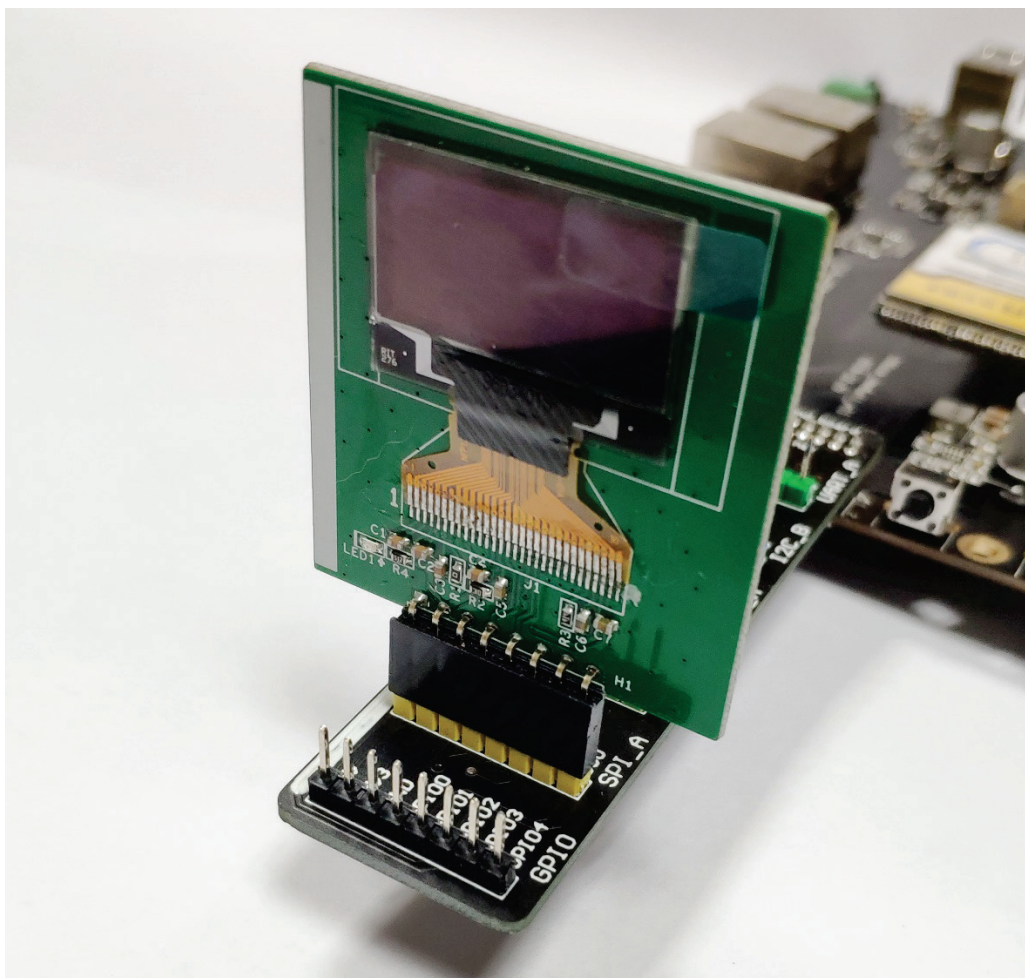


图 2.9.1 OLED 显示模块、扩展板实物连接图

2.9.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/OLED/oledDisplay  
[root@imx6ull:~]# chmod u+x oledDisplay
```

执行以下命令运行程序，即可看到 OLED 显示：

```
[root@imx6ull:~]# ./oledDisplay
```

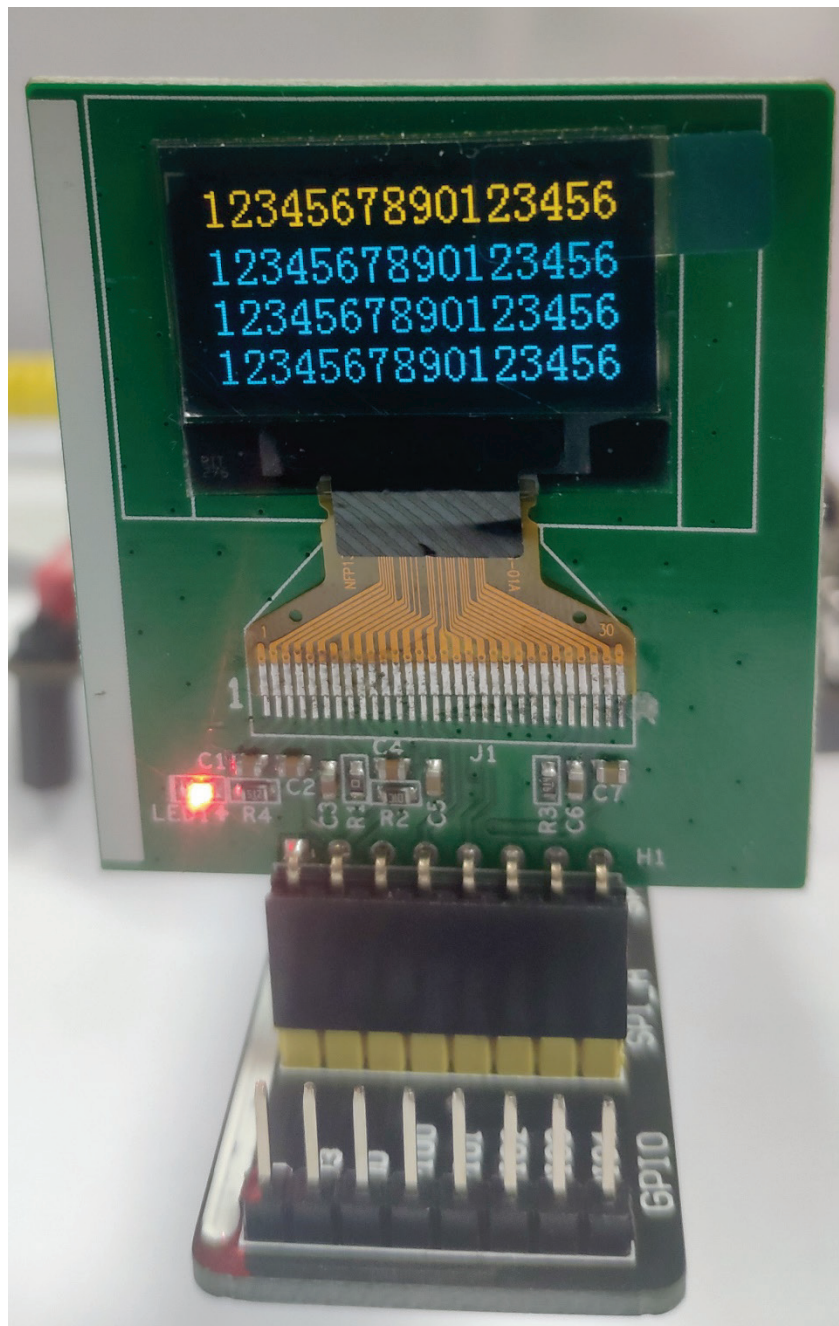


图 2.9.2 OLED 显示模块显示效果图

2.10 DAC 数模转换

前面“2.1 LED 灯”里，通过 GPIO 控制了 LED 灯的亮灭，但在生活中，有时不仅仅是控制 LED 灯的亮灭，还需要控制 LED 灯的亮度，比如常见的 LED 呼吸灯，灯光的亮度在亮和灭之间逐渐变化，感觉好像是人在呼吸。

控制 LED 灯亮度方式一般有两种，一种通过 PWM 控制脉冲的占空比，一种是输出模拟电压改变灯两端的电压，实现亮度控制。这里的 DAC 模块，程序通过 SPI 接口给 DAC 模块发送命令，DAC 模块根据命令改变输出电压，从 0V 依次到 4.092V 依次增加，实现 LED 缓缓变亮。

当然，实际项目中几乎不能用 DAC 来控制简单的 LED 灯，本例只是以 LED 灯作为最简单、直观的展示途径。

2.10.1 硬件连接

DAC 模块使用的 SPI 接口，将其插在 SPI 接口 A 上，连接实物图参考 2.10.1 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

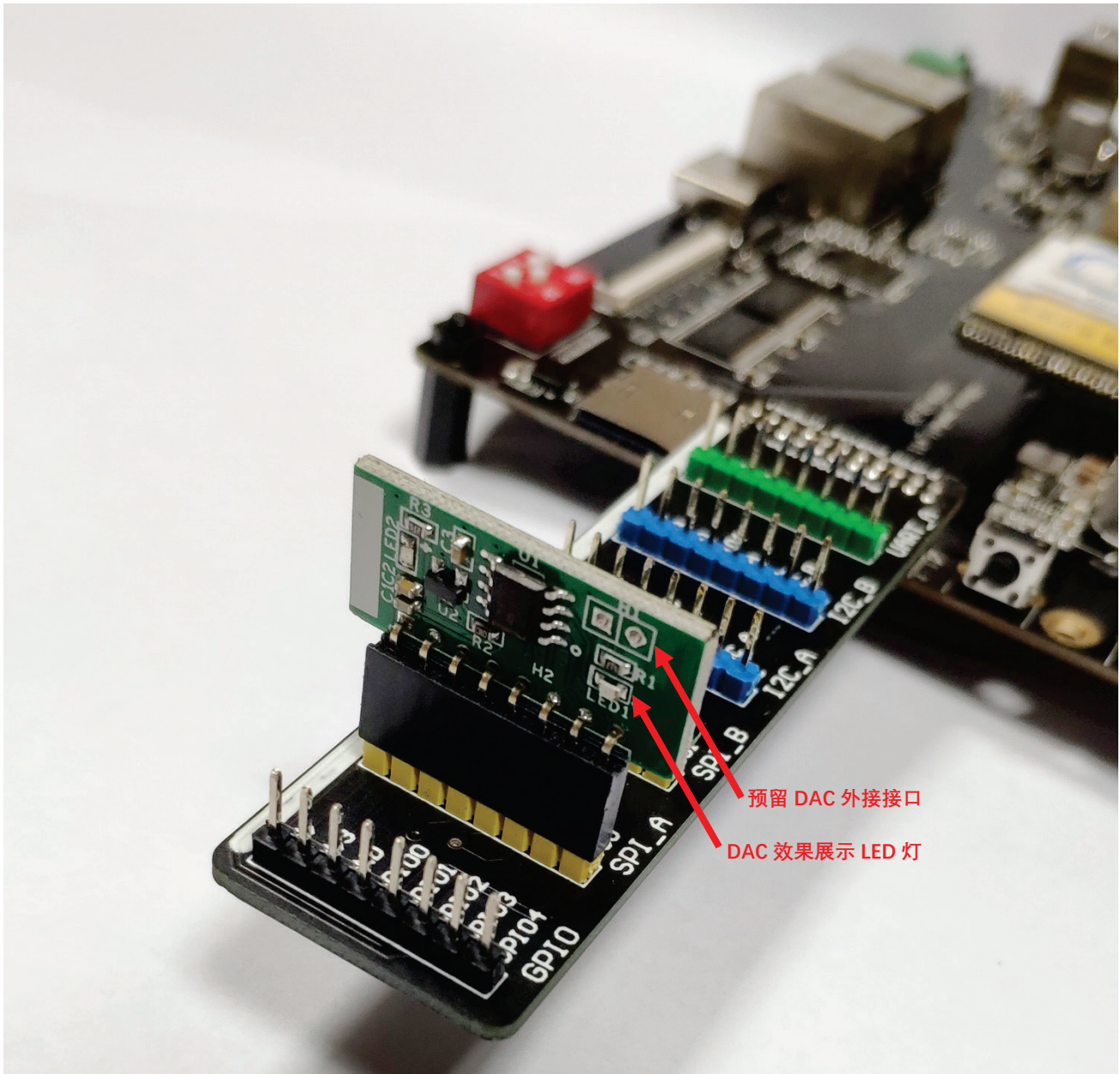


图 2.10.1 DAC 模块、扩展板实物连接图

2.10.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/DAC/breathingLight  
[root@imx6ull:~]# chmod u+x breathingLight
```

执行以下命令运行程序，即可看到 LED 灯交替缓慢变亮、变暗，类似呼吸灯效果：

```
[root@imx6ull:~]# ./breathingLight
```

2.11 ADXL345 三轴加速度计

ADXL345 是一款小而薄的超低功耗 3 轴加速度计，可以用来测量加速度，或者检测倾斜、冲击、振动等运动状态，在工业、医疗、通信、消费电子和汽车等领域中有很多应用。

2.11.1 硬件连接

ADXL345 模块使用 SPI 接口，将其插在 SPI 接口 A 上，连接实物图参考 2.11.1 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

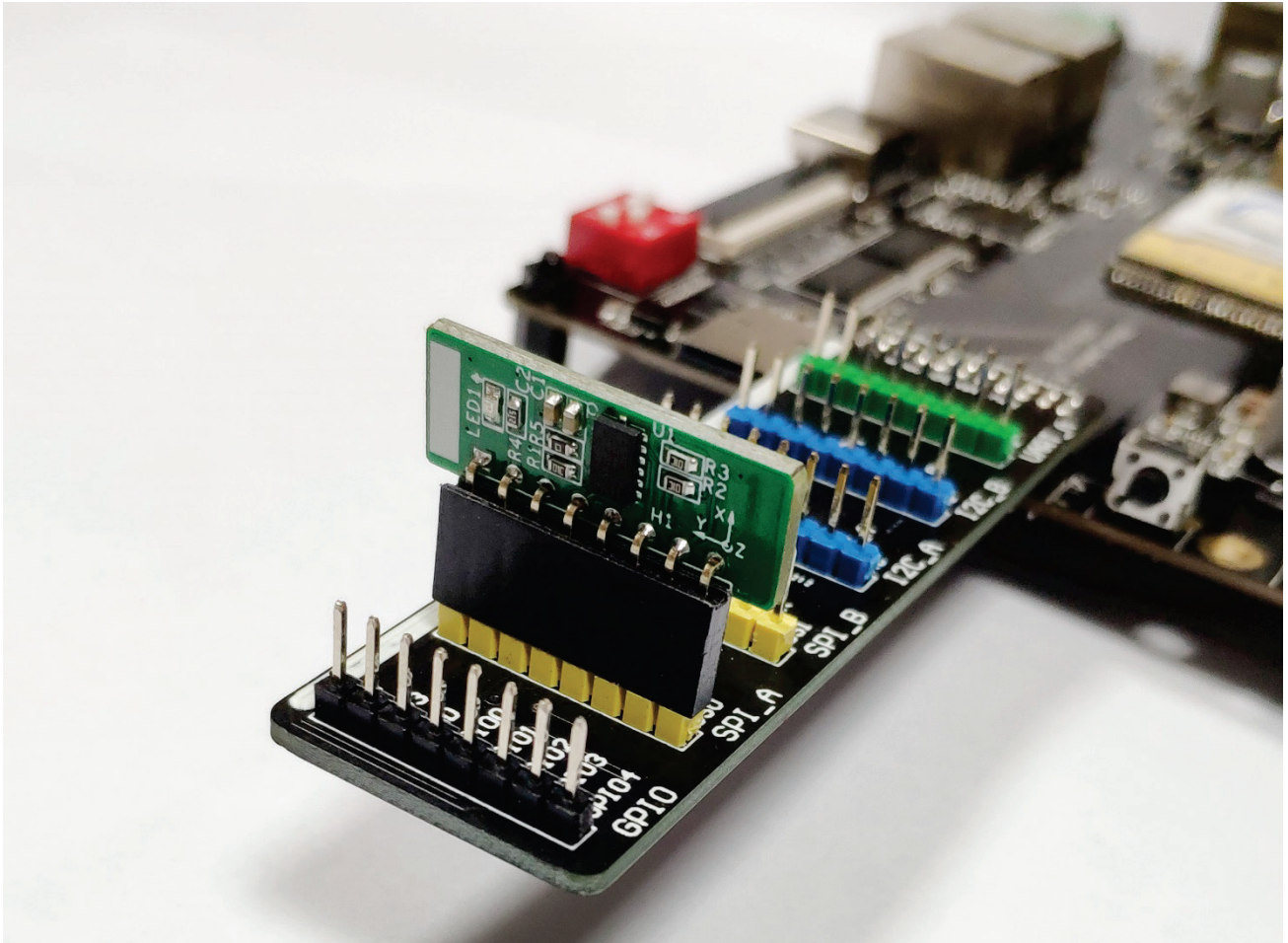


图 2.11.1 ADXL345 模块、扩展板实物连接图

2.11.2 下载、运行、效果

这里提供了两个示例程序，一个用于检测冲击，另一个用于输出 XYZ 轴的绝对位置。

以输出 XYZ 轴的绝对位置的测试为例，执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/ADXL345/readXYZData  
[root@imx6ull:~]# chmod u+x readXYZData
```

执行以下命令运行程序，当用手移动、翻转 ADXL345 模块时，会看到如下类似打印信息：


```
[root@imx6ull:~]# ./readXYZData
X: -237 Y: -87 Z: -37
X: -237 Y: -88 Z: -37
X: -237 Y: -88 Z: -39
X: -237 Y: -88 Z: -39
X: -238 Y: -88 Z: -39
X: -238 Y: -88 Z: -39
X: -238 Y: -89 Z: -39
X: -238 Y: -89 Z: -39
X: -232 Y: -89 Z: -39
```

2.12 EEPROM 存储模块

EEPROM 本质是 ROM 的一种，一般存储容量比较小，价格低廉。作为非易失性存储器，即使在掉电的状态下数据也不会丢失，常用于存储配置信息。比如将电视机中有关亮度、对比度、音量等用户个性化设置存储在 EEPROM 里，即使遇到突然停电，再打开电视时之前的设置数据也不会丢失，给用户带来了方便。

2.12.1 硬件连接

扩展板上有两个 I2C 接口，这里使用 I2C_A 接口，如图 2.12.1 所示连接。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

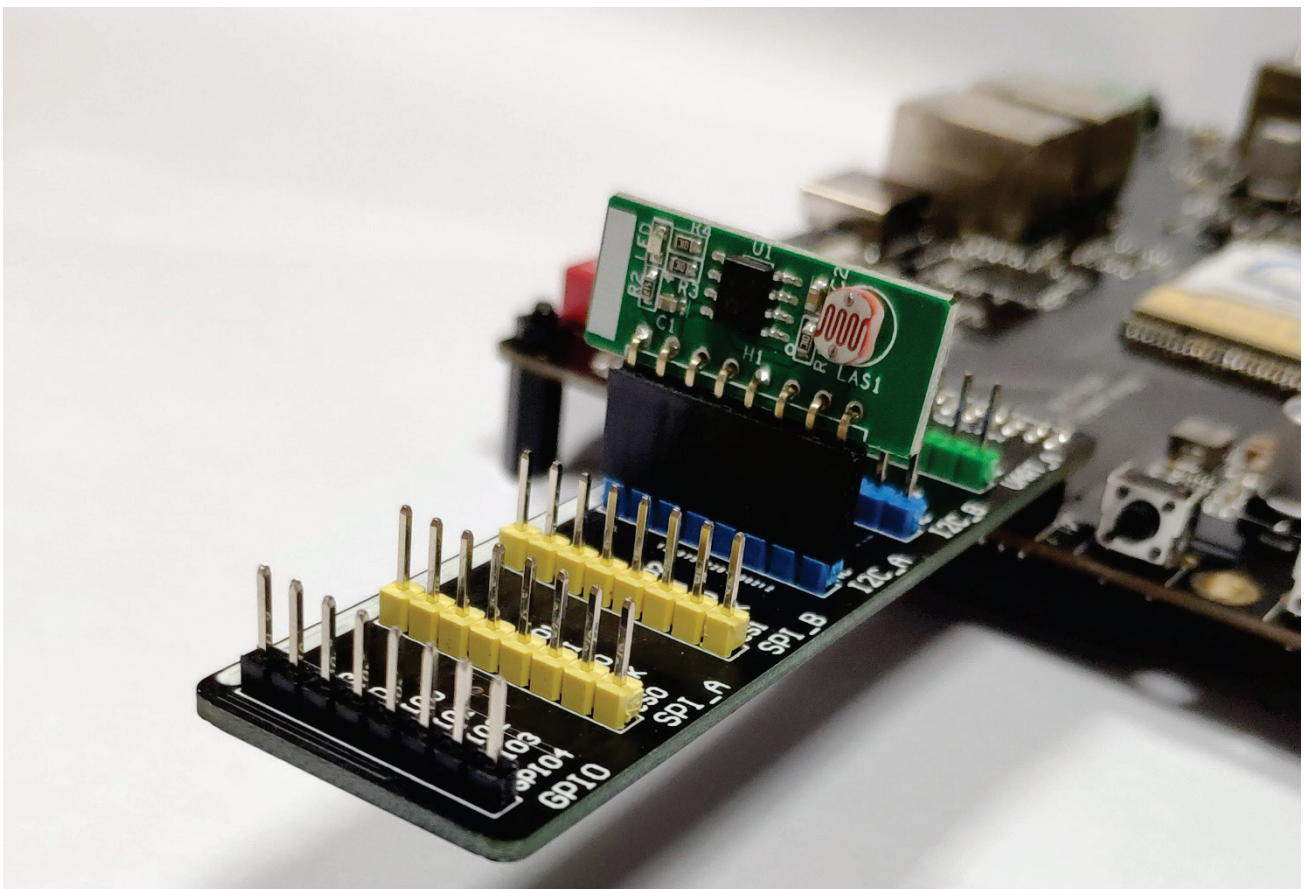


图 2.12.1 EEPROM 模块、扩展板、底板连接实物图

2.12.2 下载、运行、效果

这里提供了两个示例程序，一个以数组方式读写 EEPROM，另一个以字符串方式读写 EEPROM。

以数组方式读写 EEPROM 为例，执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/EEPROM/readWriteString  
[root@imx6ull:~]# chmod u+x readWriteString
```

执行以下命令运行程序，看到如下类似打印信息表示读写 EEPROM 成功：

```
[root@imx6ull:~]# ./readWriteString  
rxString: www.100ask.net
```

2.13 GPS 模块

串口(也称为 UART)，可以通过它发送命令给板子，也能通过它接收从板子发过来的调试信息。在开发板上不止一个串口，因此可以使用其它串口连接其它模块，收发数据。

GPS 模块使用 UART 接口上报 GPS 数据。该模块操作简单，每隔 1S 自动发送 NMEA-0183 标准的数据。

2.13.1 硬件连接

使用配套的 1 米长连接线将 GPS 模块与扩展板连接，因为 GPS 模块需要放在室外无遮挡的环境才能更好的工作，因此配套长达 1 米的连接线以方便使用。连接效果如图 2.13.1 所示。

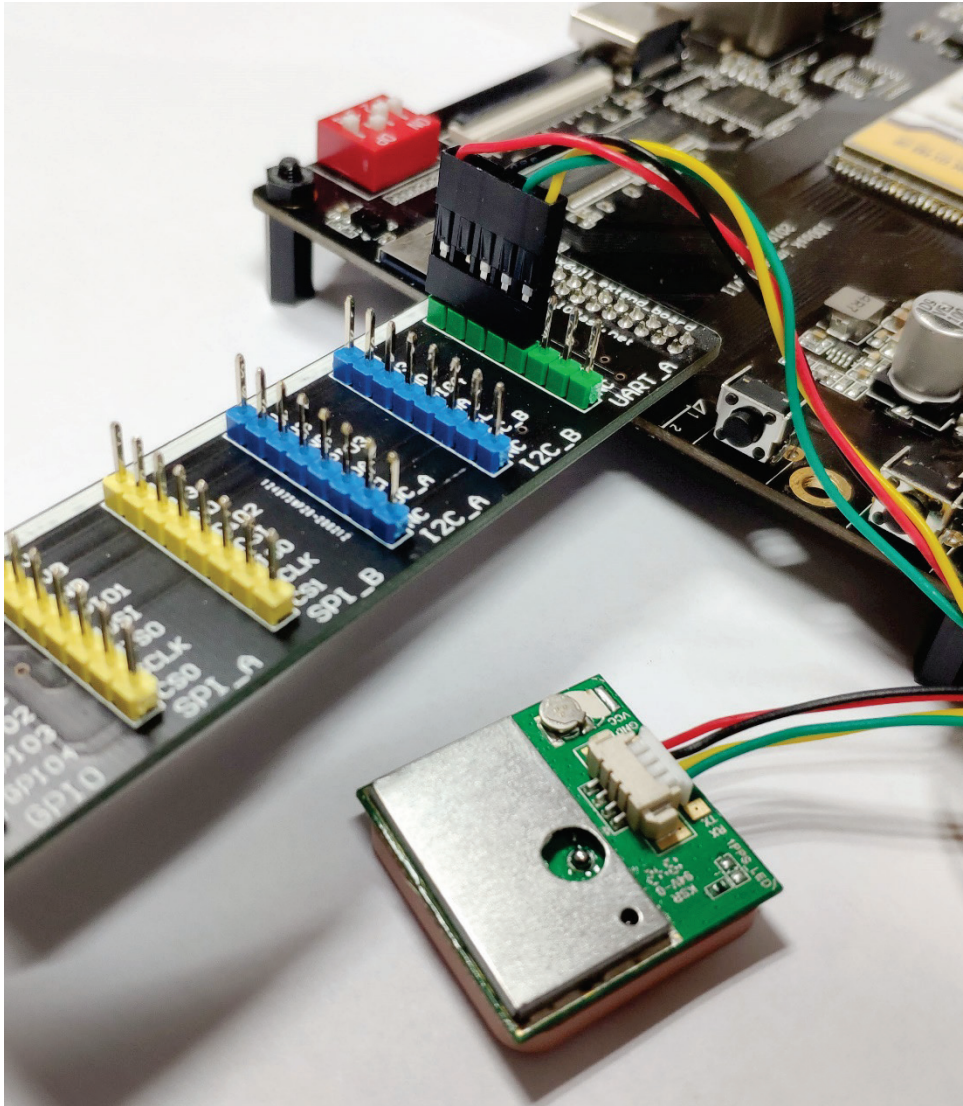


图 2.13.1 GPS 模块、扩展板、底板连接实物图

2.13.2 下载、运行、效果

这里提供了两个示例程序，一个打印 GPS 模块原始数据，另一个打印 GPS 分析处理后的数据。

以打印 GPS 分析处理后的数据为例，执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/GPS/printfAnalyzeData  
[root@imx6ull:~]# chmod u+x printfAnalyzeData
```

将 GPS 模块移至空旷开阔的地方。因为第一次是冷启动，需要重新搜索卫星定位，可能花费 3-5 分钟，效果如下：

```
[root@imx6ull:~]# ./printfAnalyzeData  
Invalid data, Please move the module to the open area and wait 3-5 minutes.  
Invalid data, Please move the module to the open area and wait 3-5 minutes.  
Invalid data, Please move the module to the open area and wait 3-5 minutes.
```

.....
22.65146 , 114.11948
22.65146 , 114.11948
22.65146 , 114.11948
22.65146 , 114.11948

打开 GPS 位置查询网址, 比如 GPSSPG 经纬度 (<http://www.gpsspg.com/maps.htm>), 输入前面经过换算的 GPS 坐标, 验证是否正确, 如图 2. 13. 2 所示。

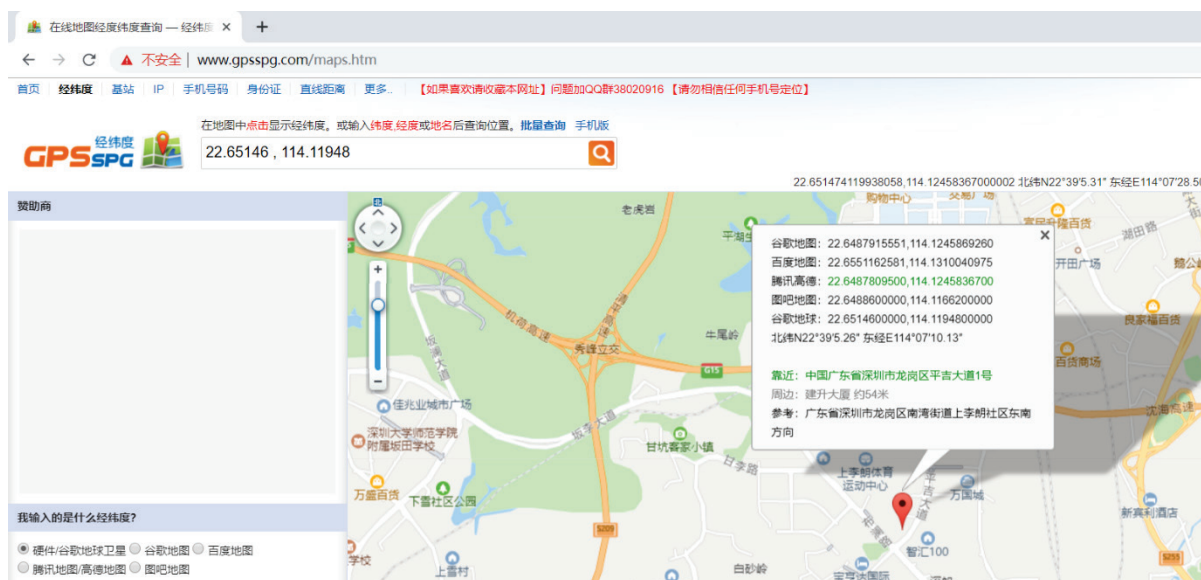


图 2. 13. 2 验证 GPS 模块数据

2. 14 ADC 模数转换

ADC (Analog-to-Digital Converter) 模数转换器, 自然界的信号几乎都是模拟信号, 比如光亮、温度、压力、声音; 而为了方便存储、处理, 计算机里面都是数字的 0/1 信号, 将模拟信号转换成数字信号的装置就叫模数转换器(ADC)。

举个例子, 光敏电阻的电路图如图 2. 14. 1 所示:

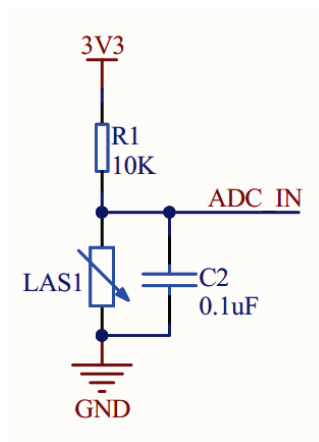


图 2. 14. 1 光敏电阻模块接线图

电路图中最上面是 3.3V 电源，往下是电阻 R1 和光敏电阻 LAS1，两个电阻之间，与 100ASK_IMX6ULL 的 ADC 引脚相连，最下面是接地，一旁的电容 C2 可以暂时忽略(它起稳压作用，防止电压突然变化)。

当光敏电阻 LAS1 受到光照时，阻值会变小，此时它两端的电压会变小；当光照比较弱时，阻值会变大，导致它两端的电压也变大。假设上图中的 ADC_IN 接到某个 GPIO 引脚，通过读 GPIO 引脚只能得到 0 或 1 两个值之一，只能判断光照在某个阈值上下。如果使用 ADC 去读取光敏电阻两端电压，不仅可以知道是否有光照，还能知道光照的强度。

几乎所有的主控芯片都有 ADC，IMX6ULL 也不例外，并且在扩展板接口上引了出来。扩展板上有两个 ADC 接口，分别在 I2C_A 所在列和 I2C_B 所在列，如图 2.14.2 所示。

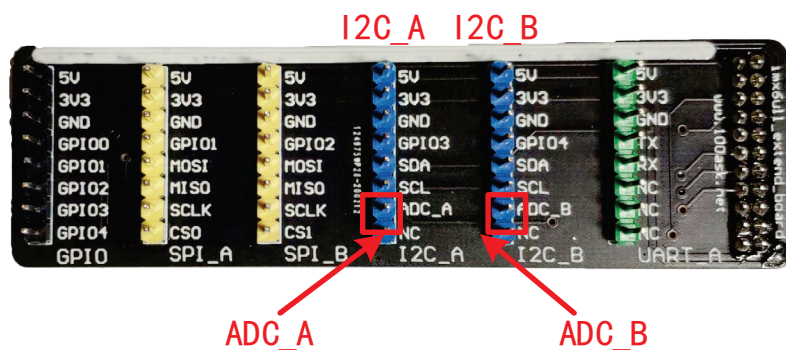


图 2.14.2 扩展板 ADC 引脚位置

光敏电阻与前面的 EEPROM 制作在了一起，直接将 EEPROM 模块接在 I2C 接口即可，如图 2.14.3 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

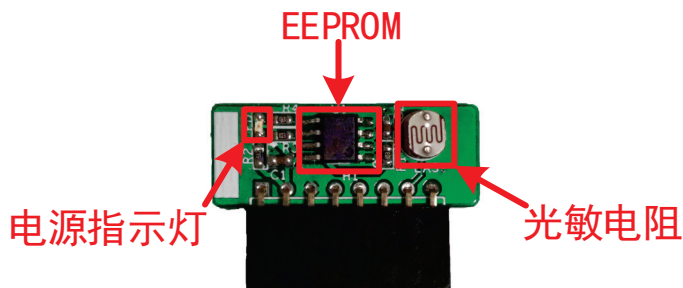


图 2.14.3 EEPROM 模块各元件位置

2.14.1 硬件连接

这里将光敏电阻模块插在 I2C_A 接口上，实物连接图如图 2.14.4 所示。

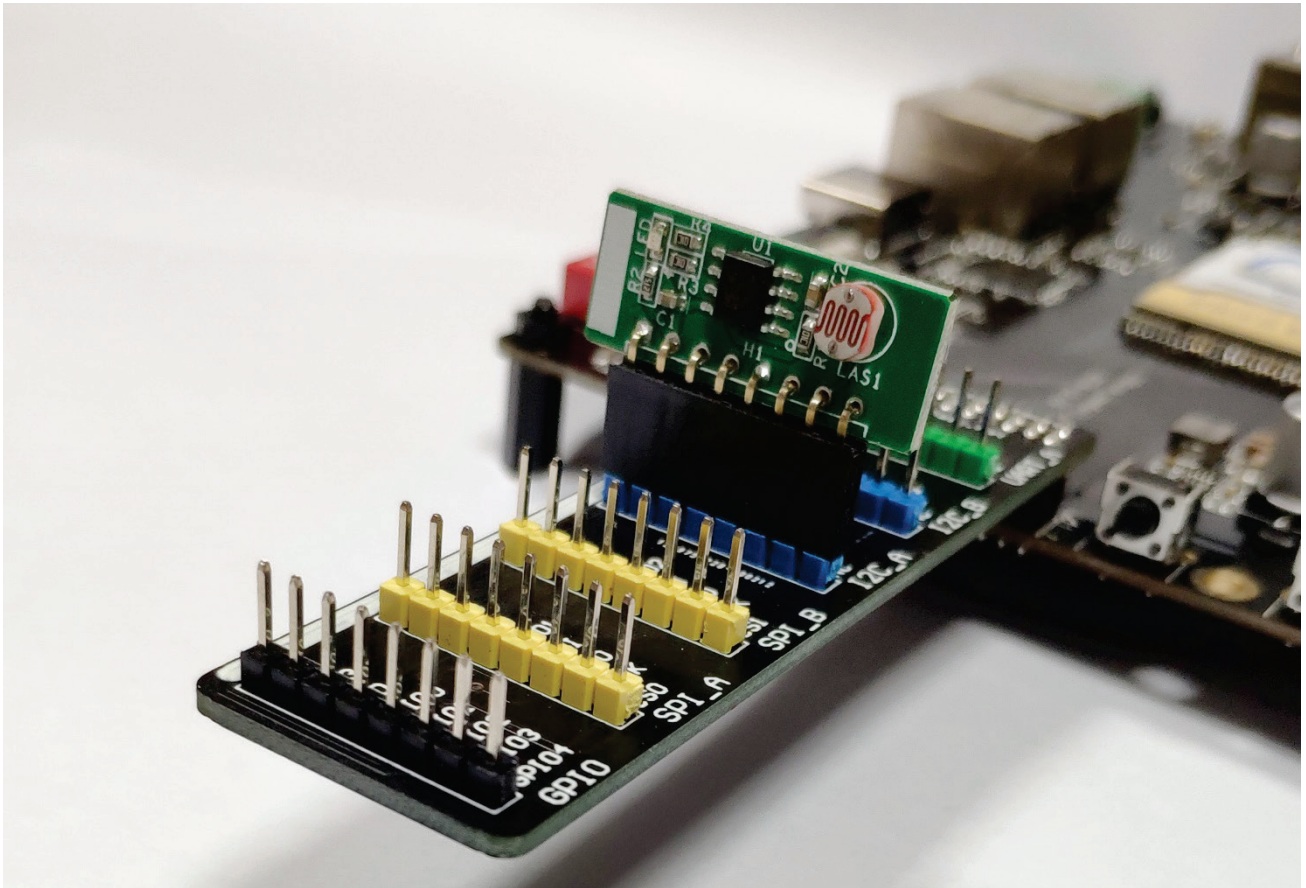


图 2.14.4 EEPROM 模块、扩展板、底板连接实物图

2.14.2 下载、运行、效果

执行如下命令下载测试文件：

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/ADC/readBrightness  
[root@imx6ull:~]# chmod u+x readBrightness
```

执行以下命令运行程序，此时用手捂住光敏电阻模块，挡住光照，可以看到电压值明显上升，效果如下：

```
[root@imx6ull:~]# ./readWriteString  
voltage: 959mv  
voltage: 960mv  
voltage: 962mv  
voltage: 961mv  
voltage: 1100mv  
voltage: 1469mv  
voltage: 1456mv  
voltage: 1458mv  
voltage: 1481mv
```

2.15 OLED 硬件监控

前面“2.5 DS18B20 温度传感器”测试里，是通过串口来显示温度结果。使用串口观察结果，一般用于调试程序。实际应用中通过电脑串口查看并不方便，因此需要换个显示途径。前面 OLED 实验里提到了，OLED 可以作为副屏显示一些简单的数据，这里刚好可以用 OLED 显示。OLED 除了可以显示外接的 DS18B20 模块产生的温度信息，也可以显示 Linux 自身信息，比如 CPU 利用率、RAM 利用率等。

本示例的内容就是通过 OLED 显示板上运行的 Linux 系统的 CPU、RAM 利用率，以及 DS18B20 温度传感器模块测出的温度值，展示多个模块协同工作。

2.15.1 硬件连接

硬件连接比较简单，只需要将 OLED 模块和 DS18B20 模块接到扩展板上，扩展板接到底板的 GPIO 扩展接口上。DS18B20 模块接在了 GPIO0 上，OLED 模块接在了 SPI_A 上，如图 2.15.1 所示。为了防止用户接错方向，模块和扩展板都有一条长白线，连接时长白线在同一侧。

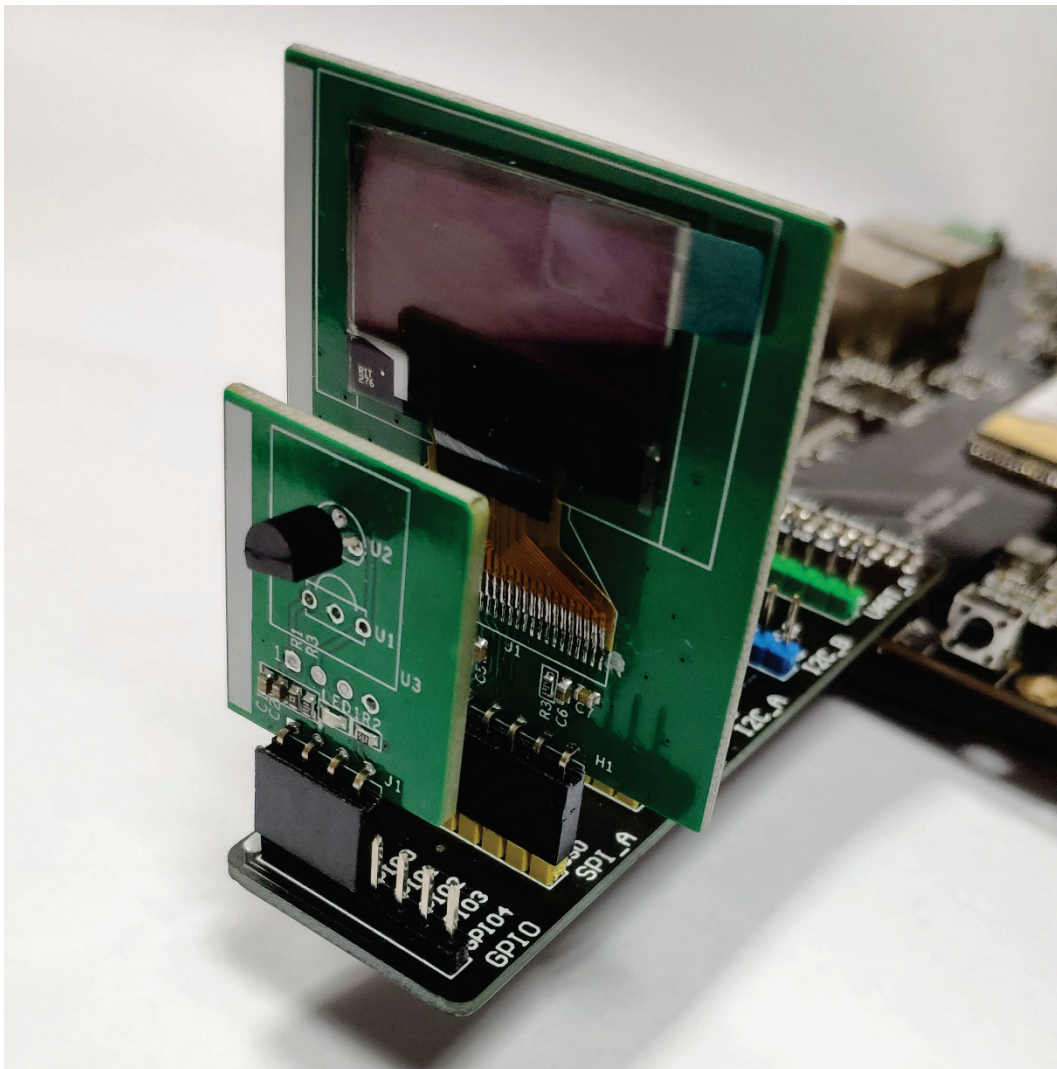


图 2.15.1 OLED 硬件监控硬件连接实物图

2.15.2 下载、运行、效果

执行如下命令下载测试文件:

```
[root@imx6ull:~]# wget --no-check-certificate  
https://weidongshan.coding.net/p/DevelopmentEnvConf/d/DevelopmentEnvConf/git/raw/master/  
imx6ullModule/app_bin/Project/OLEDMonitor  
[root@imx6ull:~]# chmod u+x OLEDMonitor
```

执行以下命令运行程序, 即可在 OLED 看到 CPU、RAM、TEP 信息, 效果如下:

```
[root@imx6ull:~]# ./OLEDMonitor
```

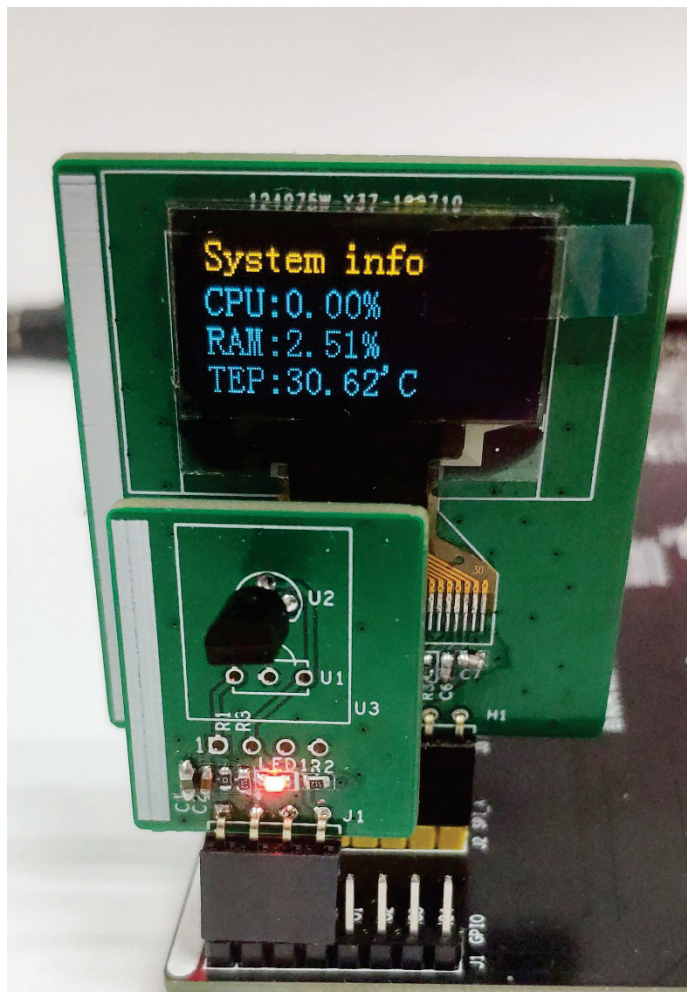


图 2.15.2 OLED 监控效果

这里看到的 CPU、RAM 利用率几乎没有变化, 可以简单的模拟一下高 CPU、RAM 负载情况, 观察数值是否变动。

- CPU 高负载测试:

```
[root@imx6ull:~] ./OLEDMonitor &  
[1] 18455  
[root@imx6ull:~]cat /dev/urandom | gzip -9 > /dev/null
```

先执行“./OLEDMonitor &”, 这里多了个“&”表示在后台执行“OLEDMonitor”, 这样才能在串口继续输入后面的命令。执行此命令后可以看到提示信息 “[1] 18455”, 它表示 “OLEDMonitor” 这个程序运行时的进程 PID (读者返回的值可能不一样), 以后可以通过此 PID 来结束此程序。

然后执行“cat /dev/urandom | gzip -9 > /dev/null”。这是一个不断获取随机数并进行压缩的操作，该过程会大量占用 CPU，使得 CPU 利用率升高。此时可以观察到 OLED 显示的 CPU 利用率变为 100%。

该测试完成后，先按下“Ctrl”+“c”，结束“cat /dev/urandom | gzip -9 > /dev/null”，再执行“kill 18455”结束后台程序“OLEDMonitor”。

- RAM 高负载测试:

```
[root@imx6ull:~] ./OLEDMonitor &  
[1] 21053  
[root@imx6ull:~] mkdir ram_test  
[root@imx6ull:~] mount -t ramfs /proc ram_test/  
[root@imx6ull:~] dd if=/dev/zero of=ram_test/file bs=1M count=128  
128+0 records in  
128+0 records out
```

先执行“./OLEDMonitor &”，让程序后台执行，系统返回该程序的 PID。

然后执行“mkdir ram_test”创建一个名为“ram_test”的目录，接着执行“mount -t ramfs /proc ram_test/”将该目录挂载到 RAM 里，最后执行“dd if=/dev/zero of=ram_test/file bs=1M count=128”向“ram_test”目录写 128M 文件，也就是向 RAM 写了 128M 文件。此时可以看到 OLED 上的 RAM 利用率明显上升，意味着 OLED 的显示反应了 RAM 的实际情况。

该测试完成后，先执行“umount ram_test/”取消挂载，再执行“rm ram_test/ -r”删除刚才新建的测试文件夹，最后执行“kill 21053”结束后台程序“OLEDMonitor”。

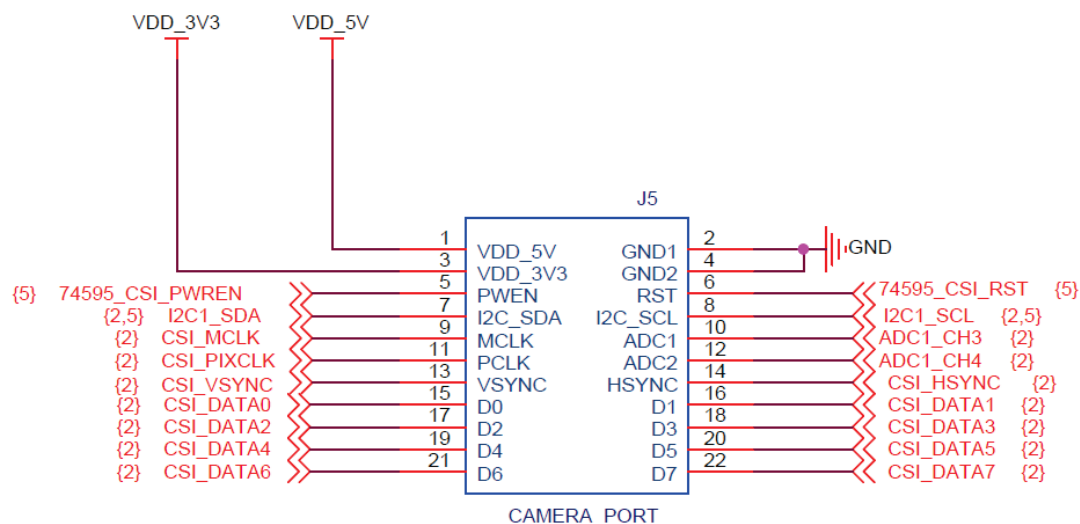
- 温度测试:

```
[root@imx6ull:~] ./OLEDMonitor
```

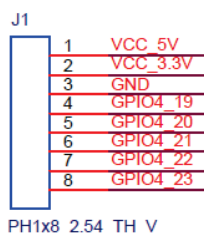
直接执行“./OLEDMonitor”，用手捏住 DS18B20 传感器，可以看到 OLED 里温度示数缓慢上升。

附录一 底板扩展接口和扩展板原理图

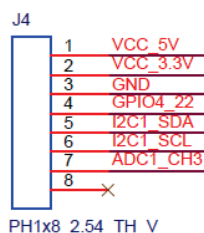
● 底板扩展接口原理图



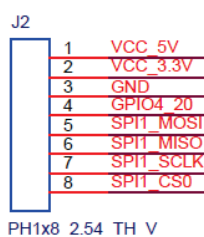
● 扩展板原理图



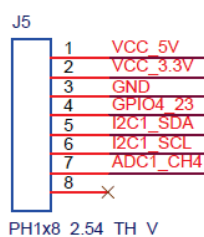
PH1x8_2.54_TH_V



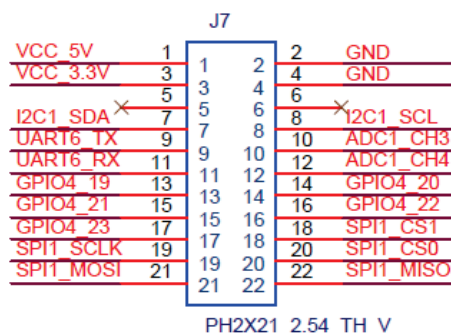
PH1x8_2.54_TH_V



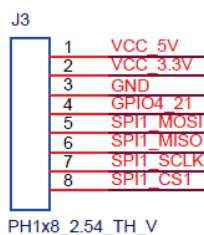
PH1x8_2.54_TH_V



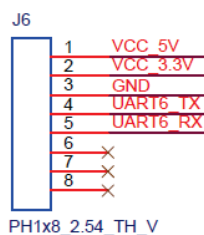
PH1x8_2.54_TH_V



PH2X21_2.54_TH_V



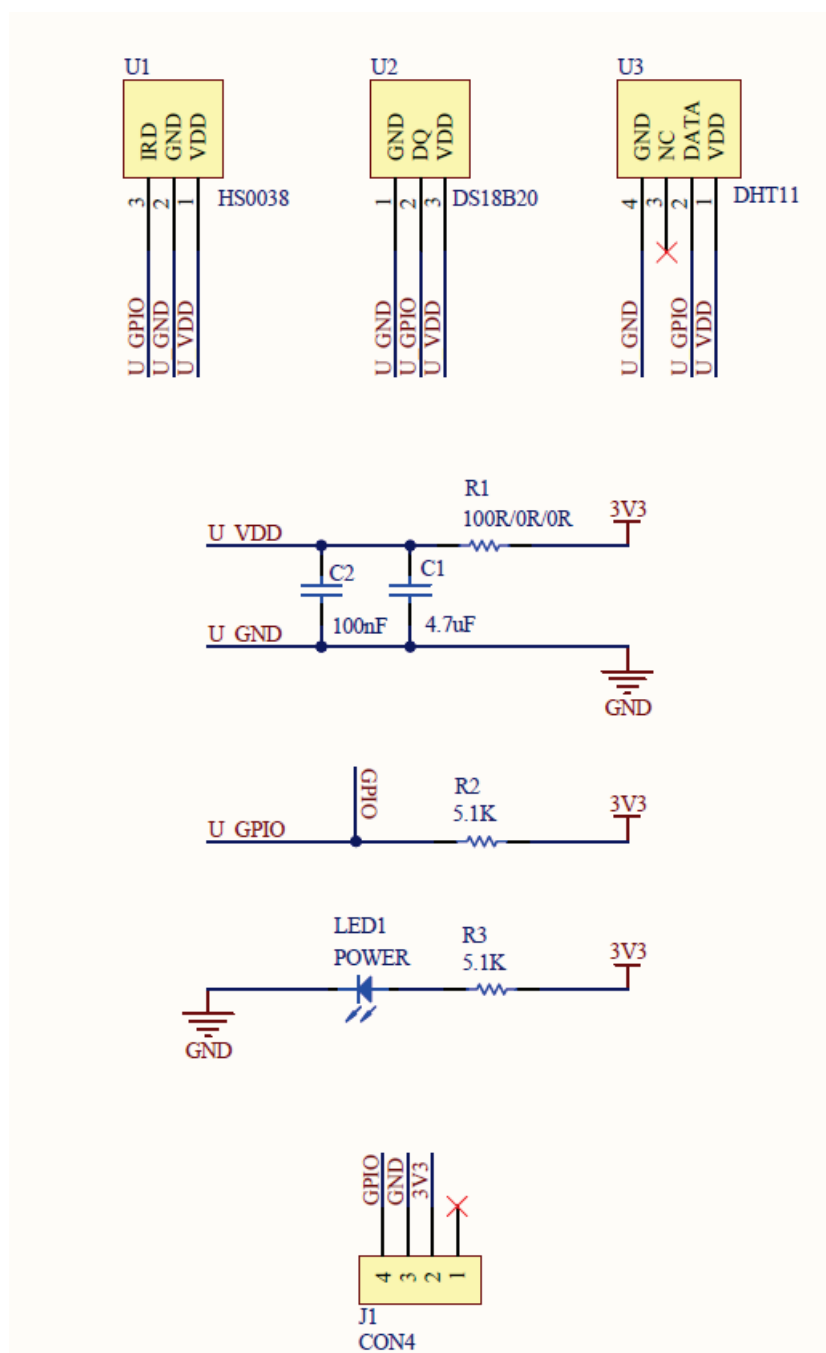
PH1x8_2.54_TH_V



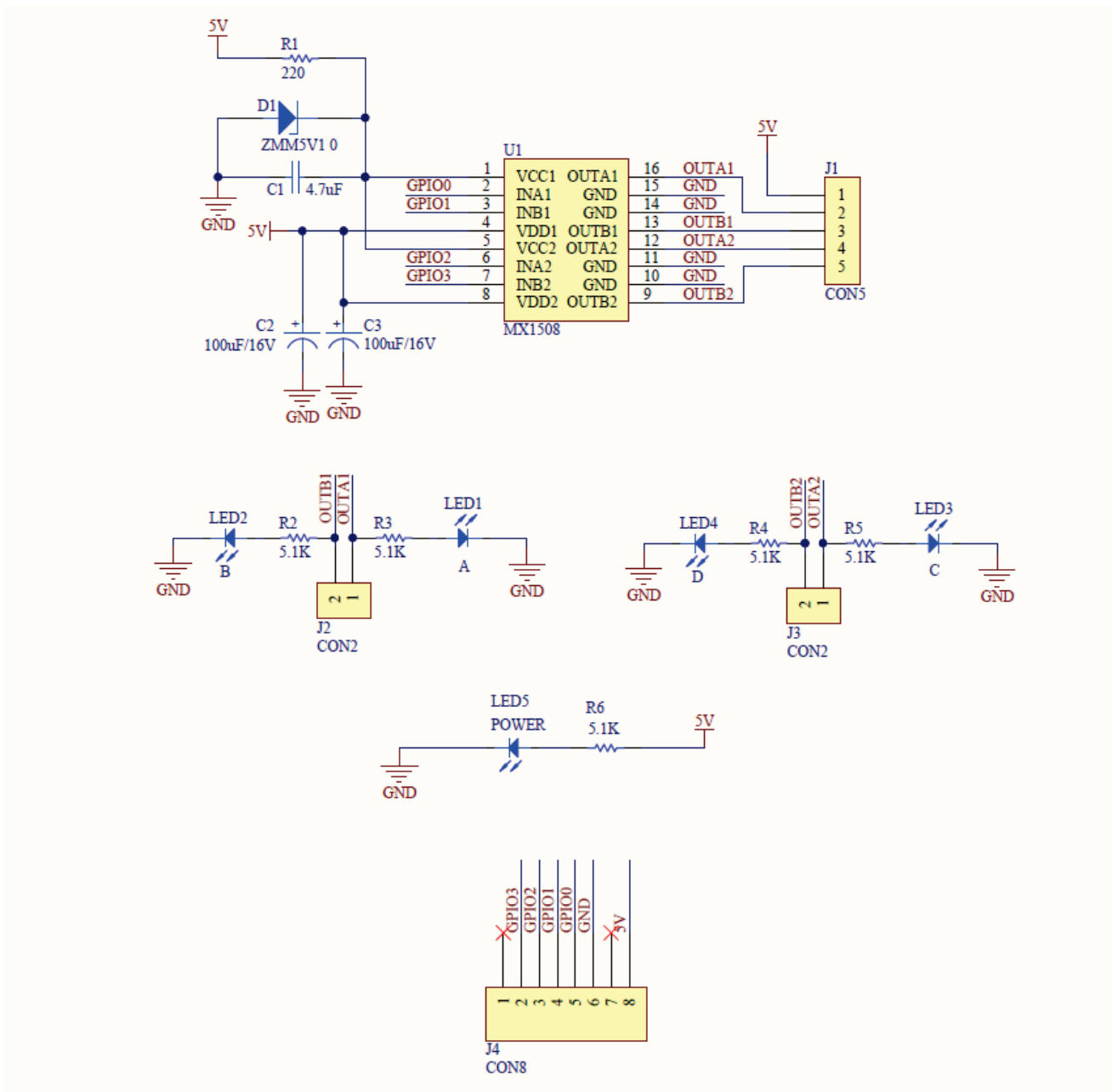
PH1x8_2.54_TH_V

附录二 模块原理图

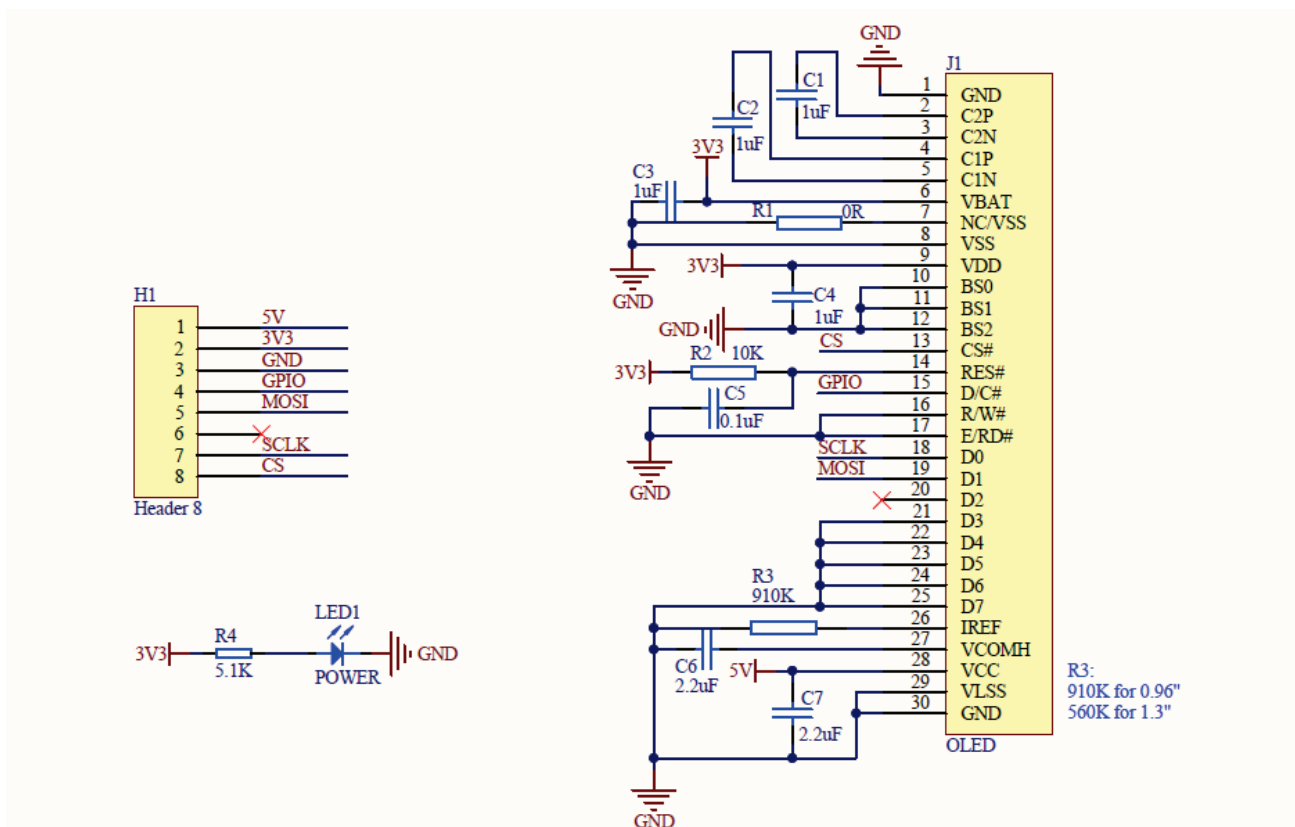
- 红外遥控接收端/DS18B20 温度传感器/DHT11 温湿度传感器原理图



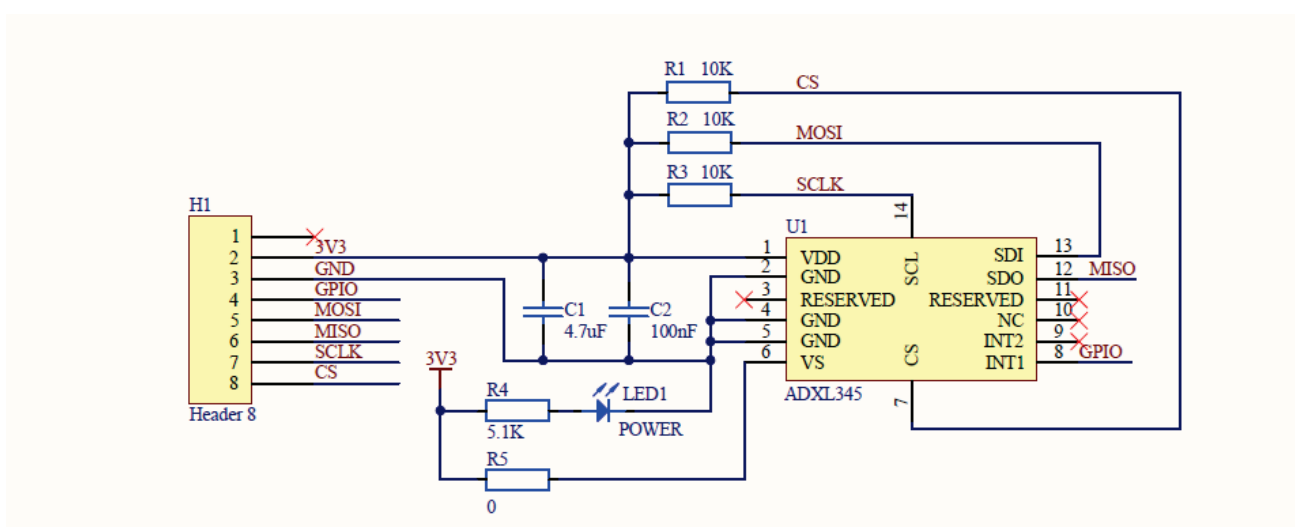
● 两路直流电机/步进电机 驱动板



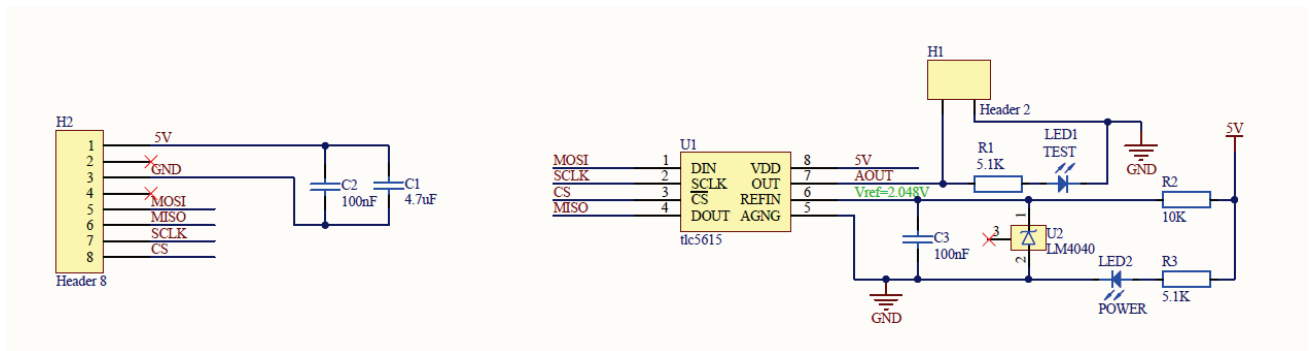
● OLED (SPI 接口) 模块原理图



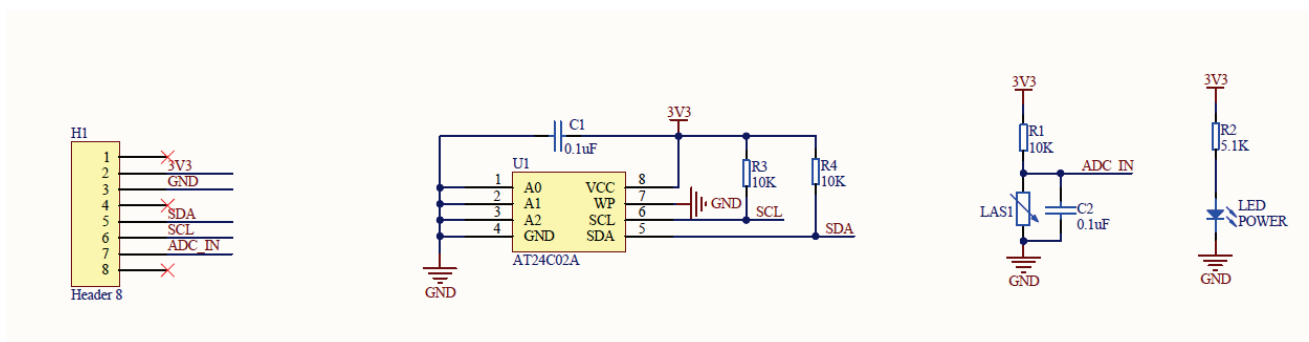
● 三轴加速度计 ADXL345 (SPI 接口) 模块原理图



● TLC5615 10Bit DAC (SPI 接口) 模块原理图



● EEPROM(I2C 接口)&光敏电阻模块原理图



附录三 按键映射表

板载按键		
名称	对应值(code)	定义
KEY1	2	KEY_1
KEY2	3	KEY_2
红外遥控器按键		
名称	对应值(code)	定义
	403	KEY_POWER
MENU	402	KEY_MENU
TEST	412	KEY_INFO
+	407	KEY_VOLUMEUP
	164	KEY_BACK
	114	KEY_PREVIOUS
	13	KEY_PLAYPAUSE
	115	KEY_NEXT
0	11	KEY_0
-	59	KEY_VOLUMEDOWN
C	60	KEY_CLEAR
1	2	KEY_1
2	3	KEY_2
3	4	KEY_3
4	5	KEY_4
5	6	KEY_5
6	7	KEY_6
7	8	KEY_7
8	9	KEY_8
9	10	KEY_9